

Umstellung Github

OSPlus-LBS 4900 Landesbausparkassen

4 August, 2026

Table of Contents

1	Zielbild für die Ablösung von Jenkins und SVN	4
1.1	1. Kurzfassung	4
1.2	2. Zielarchitektur	4
1.2.1	M/Text-Transport nach serverSync	5
1.3	3. Repositories und aktueller Entwicklungsstand	7
1.4	4. GitHub-Konfiguration	8
1.4.1	Repositories und Zugriffe	9
1.4.2	Schutzregeln für Branches, Pfade und Tags	9
1.4.3	Environments und Secrets	10
1.4.4	GitHub Actions und Ausführung	10
1.4.5	Aktualisierung der Trigger-Workflows	11
1.5	5. Branches, Weitergabe und Auslöser	11
1.5.1	Neue Releaselinie einrichten	12
1.6	6. Workflows, Trigger und Abhängigkeiten	12
1.6.1	Gesamtzusammenhang	13
1.6.2	Mandantenseitige Workflows	14
1.6.3	Zentrale Workflows	14
1.7	7. Konfigurationsdateien	15
1.7.1	config.json	15
1.7.2	mandanten.json	16
1.7.3	releaselinien.json	17
1.8	8. Release-Lieferarten FULL und DELTA	17
1.8.1	CodePipeline-Elemente	17
1.8.2	Releaseartefakt und Manifest	18
1.9	9. Mainframe-Übergabe und JCL	20
1.10	10. Status und Fehler	20
1.11	11. Qualitätsmerkmale und weitere Ausbaustufe	22
1.11.1	Tragende Qualitätsmerkmale	22
1.11.2	Mögliche Phase 2	22

2	Ablaufdiagramm Git (neu).....	24
3	Ablaufdiagramm SVN (alt)	25
4	Benutzeranleitung	26
4.1	Benutzeranleitung für die M/Text-Lieferung mit GitHub Actions	26
4.1.1	1. Zweck	26
4.1.2	2. Begriffe und Namensregeln	26
4.1.3	3. Grundlagen zu Git	27
4.1.4	4. Git-Anwendungen bedienen	31
4.1.5	5. Mandantenkonfiguration in .github/config.json	36
4.1.6	6. Neue Releaselinie einrichten	38
4.1.7	7. Änderung nach Entwicklung bringen	39
4.1.8	8. Stand zur Abnahme weitergeben	39
4.1.9	9. Ausgewählte Änderungen bereitstellen	39
4.1.10	10. FULL- oder DELTA-Lieferung auslösen	40
4.1.11	11. Einen Lauf kontrolliert wiederholen	41
4.1.12	12. Status und Fehlerbilder verstehen	42
5	GitHub Repositories, Teams und IDNeo Fachrollen	44
5.1	Allgemein.....	44
5.2	Übersicht	44
5.3	GitHub Einrichtung	46
6	Nächste Schritte (OPL)	47
6.1	1. Technische Grundlagen festlegen	47
6.2	2. Runnerangebot der FI bereitstellen und prüfen	47
6.3	3. GitHub-Einstellungen und Berechtigungen einrichten.....	48
6.4	4. Workflowdateien einrichten und aktualisieren.....	49
6.5	5. M/Text-Transport entscheiden und abnehmen	50
6.6	6. Mainframe-Übergabe einrichten und abnehmen	51
6.7	7. Git-Client und Bedienabläufe abnehmen	51
6.8	8. Test-Parallelbetrieb vorbereiten.....	52

1 Zielbild für die Ablösung von Jenkins und SVN

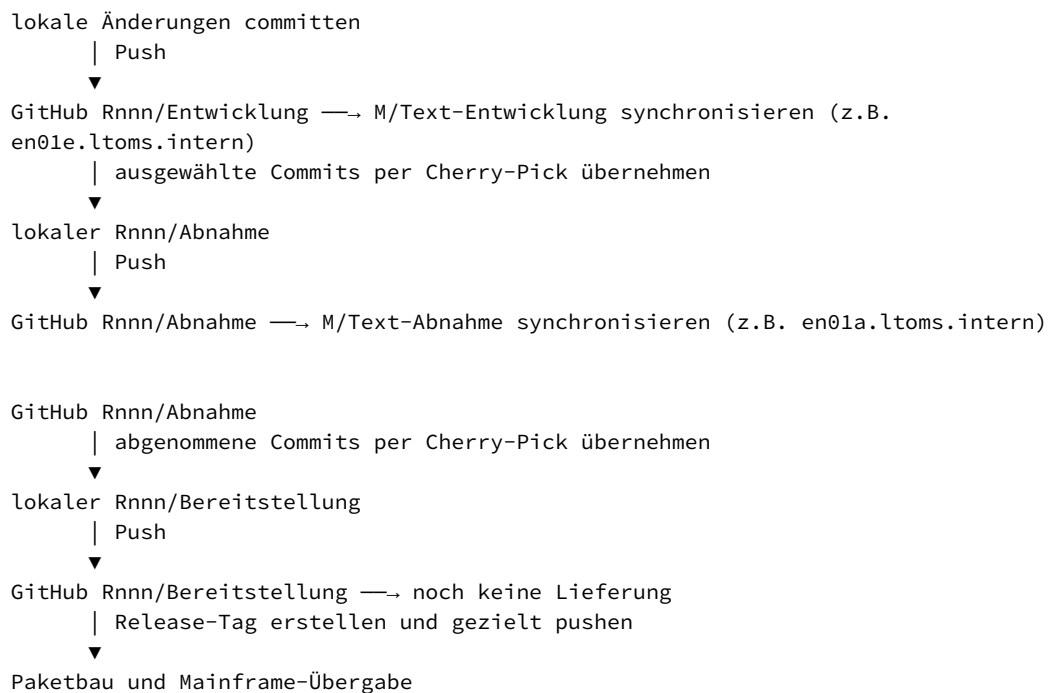
1.1 1. Kurzfassung

SVN wird durch Git und Jenkins durch GitHub Actions ersetzt. Dafür wird voraussichtlich ab November oder Dezember 2026 ein SVN-Abzug nach Git übernommen und in GitHub verprobt. Während dieser Testphase bleibt der bisherige Prozess produktiv. Unmittelbar vor der für Januar 2027 geplanten Produktivsetzung wird der dann freigegebene SVN-Endstand nach Git übertragen. Danach sind Git und GitHub Actions für diesen Prozess führend und SVN wird zusammen mit der EN4920 abgebaut.

Jeder Mandant erhält ein eigenes Repository mit seinen M/Text-Ressourcen und seiner Konfiguration. Die gemeinsame Automatisierung liegt im zentralen Repository `mtext-actions`. So bleiben die Mandantendaten getrennt, während alle Mandanten dieselben geprüften Abläufe verwenden.

Eine Änderung durchläuft weiterhin die drei Prozess-Stages Entwicklung, Abnahme und Bereitstellung für die Lieferung.

Der generelle Ablauf ist:



1.2 2. Zielarchitektur

Die Lösung besteht aus vier Bereichen:

Bereich	Verantwortung
Mandanten-Repository	M/Text-Ressourcen, Mandantenkonfiguration und Workflow-Trigger
Automatisierungs-Repository <code>mtext-actions</code>	Prüfungen, Synchronisation, Paketbau und Mainframe-Übergabe
GitHub Actions	Ausführung der Abläufe, Protokollierung
GitHub-Actions-Runner (FI)	Ausführung der Workflows auf dem offiziell verfügbaren Runnerangebot der FI

Ein Mandanten-Repository enthält seine M/Text Brief-Ressourcen und die für GitHub relevante Konfiguration. Das zentrale Automatisierungs-Repository `mtext-actions` enthält nur die gemeinsam verwendete Automatisierung.

Mandanten-Repositories dürfen die wiederverwendbaren Workflows über eine GitHub-Actions-Zugriffsrichtlinie aufrufen. Die Logs der aufgerufenen Jobs sind im Mandanten-Repository sichtbar und enthalten deshalb weder Secrets noch unnötige interne Details.

Die Administratoren legen die Berechtigungsregeln für GitHub Actions fest. GitHub erzeugt für jeden Job ein `GITHUB_TOKEN` mit den daraus resultierenden Rechten.

Die Text-Entwickler bearbeiten Briefressourcen in der M/Text Workbench und nutzen den integrierten Git-Client für alle Aktionen, inkl. Cherry-Picking oder das Erstellen von Release-Tags. GitHub im Browser dient für Laufkontrolle, Wiederholungen und die Prüfung von Release-Tags. Für die tägliche Arbeit ist keine Git-Kommandozeile nötig.

Jeder Mandant benennt ein Mandanten-Release-Team für den Bereitstellungsbranch und die Release-Tags. Die Verantwortlichen können sich je Mandant unterscheiden.

Für jeden Lauf checkt GitHub Actions sowohl den ausgewählten Stand des Mandanten-Repositories als auch eine festgelegte `mtext-actions`-Version aus. Dadurch ist später nachvollziehbar, welche Quellen und welche Automatisierung tatsächlich verwendet wurden.

1.2.1 M/Text-Transport nach `serverSync`

Der heutige Ablauf stellt die zu synchronisierenden Ressourcen zuerst über ein NFS-Share unter `serverSync` bereit. Danach sendet er einen POST-Request an den M/Text-Adapter (LTOMA), der die interne Synchronisation mit dem M/Text-Server (LTOMS) anstößt. Dieser Ablauf ist die Ausgangslage. Der künftige Transportweg ist noch nicht festgelegt.

Unabhängig vom Transportweg entsteht für jedes synchronisierte Projekt auf `serverSync` derselbe vollständige Verzeichnisbaum mit denselben relativen Pfaden, Dateinamen und Dateinhalten wie im bisherigen Jenkins-/SVN-Verfahren. Der Workflow bereitet diesen Stand zunächst in einem

temporären Staging-Verzeichnis des Runners vor. Die drei Transportwege verwenden damit denselben fachlichen Inhalt.

Für die Übertragung per PUT oder über den Artefaktspeicher von GitHub Actions erzeugt der Workflow aus dem Staging-Verzeichnis ein ZIP-Transportpaket, in dem direkt die Projektverzeichnisse liegen.

```
serverSync-R261-Entwicklung-<Commit-SHA>.zip
  LOMS_Basis[BY]/
    <vollständiger Projektbaum>
  LOMS_Autonom[BY]/
    <vollständiger Projektbaum>
```

Dasselbe ZIP-Transportpaket dient als PUT-Nutzlast oder als Inhalt eines GitHub-Actions-Artefakts. Ein GitHub-Actions-Artefakt speichert Dateien eines Workflow-Laufs im GitHub-Artefaktspeicher. Der Workflow legt seinen Namen, seine Aufbewahrungsfrist und die darin gespeicherten Dateien fest. GitHub vergibt unter anderem die Artefakt-ID und regelt Speicherung, Berechtigung und Download. Der Download erfolgt über die GitHub-API. LTOMA entpackt die GitHub-Verpackung und erhält daraus das originäre ZIP-Transportpaket.

Die Veröffentlichung erfolgt erst, nachdem der gesamte Projektstand erfolgreich übertragen wurde. Mehrere Mandanten-Repositories teilen sich `serverSync`.

Für die Versorgung von `serverSync` werden drei mögliche Varianten geprüft:

Variante	Ablauf	Zu klären	Implementierungsaufwand
PUT an den Adapter	Der Workflow erzeugt das ZIP-Transportpaket und sendet diese Datei als Payload eines PUT-Requests an den Adapter. Der Adapter prüft und entpackt es, veröffentlicht die enthaltenen Projektverzeichnisse unter <code>serverSync</code> und startet die interne Synchronisation.	HTTP-Vertrag, Authentifizierung, Größenlimits, Prüfsummen, Zeitgrenzen, Wiederholung, Parallelität und Erfolgsstatus	mittel bis hoch

Variante	Ablauf	Zu klären	Implementierungs-Aufwand
Direkter Sharezugriff des Runners	Der Runner stellt den vorbereiteten Verzeichnisbaum direkt auf dem NFS-/Netzlaufwerk des M/Text-Servers bereit und ruft erst danach den Adapter auf. Das ZIP-Transportpaket wird für diesen Weg nicht benötigt. Staging, Veröffentlichung und Wiederanlauf liegen damit in der GitHub-Automatisierung. Diese Variante entspricht dem aktuellen Entwicklungsstand.	Verfügbarkeit und Einbindung des Shares, Pfad, Rechte, Kapazität, atomare Ersetzung, Schutz vor parallelen Schreibvorgängen und Bereinigung nach Fehlern	gering
Download aus dem Artefaktspeicher von GitHub Actions	Der Workflow erzeugt dasselbe ZIP-Transportpakete wie in der PUT-Variante, und lädt es als Datei in den GitHub-Actions-Artefaktspeicher ("Actions-Artefakt"). LTOMA (oder LTOMS) lädt diese Datei herunter, prüft und extrahiert sie unter <code>serverSync</code> und startet die interne Synchronisation wie gehabt.	Zuständige Zielkomponente, eindeutige Identifikation durch Repository, Workflow-Lauf und Artefakt-ID, technische Identität mit <code>Actions: read</code> , Prüfsumme, Erreichbarkeit, Aufbewahrungsfrist, Wiederholung und Bereinigung	mittel

Vor dem nichtproduktiven Integrationslauf wird genau eine Variante ausgewählt. Die Entscheidung berücksichtigt Netzwerk- und Sicherheitsvorgaben, Betriebsverantwortung, Datenmengen und Laufzeiten, atomare Veröffentlichung, Parallelität, Wiederanlauf und Nachvollziehbarkeit. Implementiert wird nur der ausgewählte Weg.

1.3 3. Repositories und aktueller Entwicklungsstand

Ein Mandanten-Repository folgt diesem Grundaufbau:

```

mtext-<mandant>/
  .github/
    config.json
    workflows/
      validate-config.yml
      sync-resources.yml
      release.yml
  <M/Text-Projekte>

```

`mtext-fi` dient als Muster für die Mandanten-Repositories. Alle sichtbaren Verzeichnisse in der Repositorywurzel werden synchronisiert und in Releasepakete aufgenommen. Es ist möglich

Verzeichnisse (z.B. LOMS_Testdaten) via `config.json` von der Synchronisation und den Releasepaketen auszuschließen, aber trotzdem Bestandteil des von Git verwalteten Repositories zu sein. Mittels `.gitignore` ist es möglich Dateien generell auszuschließen.

Im Mandanten-Repository stehen nur kleine Trigger-Workflows. Die eigentlichen Arbeitsschritte liegen in `mtext-actions`. Bei der Einrichtung (und bei späteren Updates) informiert der `update-mandant-workflow` die Trigger-Workflows darüber, welche Version von `mtext-actions` zu verwenden ist.

Das zentrale CI/CD Repository enthält die wiederverwendbaren Workflows, die gemeinsame Python-Anwendung, die zentrale Releaselinien- und Mandantenzuordnung, das JCL-Template und die Tests:

```
mtext-actions/
  .github/
    workflows/
      ci.yml
      reusable-release.yml
      reusable-sync-resources.yml
      reusable-validate-config.yml
      update-mandant-workflow.yml
  config/
    mandanten.json
    releaselinien.json
  scripts/
    runner-preflight.sh
  src/
    build_release.py
    publish_mainframe.py
    sync_resources.py
    validate_config.py
    workflow_configuration.py
    lbs_delivery/
  templates/
    mainframe-upload.jcl
  tests/
```

Hier ist `lbs_delivery` das zentrale Python Modul, das diverse Funktionalitäten (Error-Codes, Git Funktionen, Paketbau, ...), die von den eigentlichen Workflow-Skripten benutzt werden, kapselt.

Vor dem ersten Integrationslauf werden Runner-Kennzeichen und zentrale Workflowversion finalisiert. Die noch ausstehenden Einrichtungs- und Abnahmepunkte stehen in [Nächste Schritte \(see page 4\)](#).

1.4 4. GitHub-Konfiguration

Für Planung und Abnahme ist GitHub Enterprise Server (>= 3.20.4) als Zielplattform festgelegt. Die folgenden Einstellungen definieren den gewünschten Zielzustand.

1.4.1 Repositories und Zugriffe

Gegenstand	Zielzugriff
Mandanten-Repositories	Für jeden Mandanten besteht ein eigenes Repository
Zentrales Repository	<code>mtext-actions</code> ist privat und nur für das zentrale Automatisierungsteam der FI direkt zugänglich
Default Branch	Der Entwicklungsbranch der führenden Releaselinie – zunächst <code>R261/Entwicklung</code> – wird beim Wechsel der führenden Linie angepasst
Mandanten-Release-Team	Der benannte Verantwortlichenkreis, der die Mandantenkonfiguration in <code>.github/config.json</code> ändern, nach <code>Rnnn/Bereitstellung</code> pushen sowie Release-Tags anlegen darf

1.4.2 Schutzregeln für Branches, Pfade und Tags

Schutzbereich	Regel
<code>Rnnn/Entwicklung</code> und <code>Rnnn/Abnahme</code>	Berechtigte Text-Entwickler dürfen pushen. Force-Pushes und das Löschen der Branches sind gesperrt.
<code>Rnnn/Bereitstellung</code>	Reguläre Pushes sind auf das Mandanten-Release-Team begrenzt. Force-Pushes und das Löschen des Branches sind gesperrt.
<code>.github/workflows/**/*</code>	Ein Push-Ruleset schützt die zentral vorgegebenen Aufrufdateien auf allen Branches.
<code>.github/config.json</code>	Eine Pfadregel trennt Änderungen der Mandantenkonfiguration von der normalen Ressourcenpflege.
Tags <code>Rnnn.nnn</code>	Nur das Mandanten-Release-Team darf passende Tags erstellen oder löschen.

1.4.3 Environments und Secrets

Ein GitHub Environment bildet eine Zielstufe ab und enthält alle dafür geltenden Schutzregeln, etwa zulässige Branches oder Tags und erforderliche Freigaben. Seine Secrets stehen Jobs zur Verfügung, die an dieses Environment gebunden sind und dessen Schutzregeln erfüllen.

Environment	Verwendung und Schutz
Einrichtung	Wird ausschließlich vom manuell gestarteten Einrichtungsworkflow in <code>mtext-actions</code> gebunden. Es stellt nach Freigabe das auf die vorgesehenen Repositories begrenzte technische Token für die Workflowänderungen bereit.
Bereitstellung	Wird ausschließlich vom Publish-Job gebunden. Eine berechnigte Person muss die Mainframe-Übergabe manuell freigeben. Ein verpflichtendes Vier-Augen-Prinzip oder eine Sperre der Selbstfreigabe ist nicht vorgesehen. Nur zulässige Release-Tags dürfen dieses Environment verwenden.

Die Mainframe-

Zugangsdaten `MAINFRAME_FTP_HOST`, `MAINFRAME_FTP_USER` und `MAINFRAME_FTP_PASSWORD` liegen als Secrets im Environment `Bereitstellung`. Sie werden weder in Git gespeichert noch an den Build-Job übergeben.

1.4.4 GitHub Actions und Ausführung

Nach abgeschlossener Einrichtung gelten für die Ausführung der Workflows die folgenden technischen Festlegungen:

Gegenstand	Ergebnis der Einrichtung
Zentrale Workflowversion	Jeder Aufruf verwendet die für seinen Rollout festgelegte Version von <code>mtext-actions</code>
Aktualisierungsberechtigung	Der Mandanten-Aktualisierungsworkflow erhält über das Environment <code>Einrichtung</code> das Secret <code>WORKFLOW_CONFIGURATION_TOKEN</code>
Runnerangebot der FI	Die Jobs verwenden einen offiziell von der FI bereitgestellten GitHub-Actions-Runner. Das zugehörige <code>runs-on</code> -Kennzeichen wird aus dem Runnerangebot der FI übernommen und in den zentralen Workflows fest eingetragen
Laufzeitvorbereitung	<code>runner-preflight.sh</code> ist der gemeinsame Einstieg in die Python-Automatisierung. Es setzt die versionierte Laufzeitvorgabe aus <code>.python-version</code> durch - dadurch laufen alle Workflows mit derselben technischen Voraussetzung

Gegenstand	Ergebnis der Einrichtung
Logs	Ausgaben wiederverwendbarer Workflows sind im Mandanten-Repository sichtbar
Artefakte	Releaseartefakte werden standardmäßig 30 Tage aufbewahrt. Ihre Namen enthalten Repository und Release-Tag

1.4.5 Aktualisierung der Trigger-Workflows

Ein neuer Commit in `mtext-actions`, z.B. weil sich die Releaselinien ändern, ist nicht automatisch in den Mandanten-Repositories verfügbar, da deren Trigger-Workflows erstmal auf eine vorherige Version zeigen. Der Commit wird deshalb über einen manuell zu startenden Batchlauf (**Update mandant workflows**) auf alle in `config/mandanten.json` konfigurierten Mandantenrepositories und deren Branches (i.d.R. sind dies $7 \times 3 \times 3 = 63$ Branches) verteilt. Bei der Initialisierung der Mandanten-Repositories zeigen deren Trigger-Workflows noch auf nichts (Null-SHA) und müssen daher ebenfalls durch diesen Batch-Workflow einmal eingerichtet werden.

Vor dem ersten Lauf muss im Repository `mtext-actions` das `runs-on`-Kennzeichen des FI-Runners als Repository-Variable `FI_RUNNER_LABEL` hinterlegt werden. Dort wird außerdem das Environment `Einrichtung` eingerichtet und darin das Secret `WORKFLOW_CONFIGURATION_TOKEN` hinterlegt. Das Token gehört zu einer technischen Identität, die Workflowdateien in `mtext-actions` und den Mandanten-Banches festschreiben darf. Dann wird der Batch-Workflow in GitHub manuell gestartet. Als Eingabe wird die SHA des auszurollenden Commits angegeben.

Der Mandanten-Commit ändert nur `.github/workflows` und löst wegen seiner Skip-Anweisung keine M/Text Synchronisation oder Releaseverarbeitung aus. Fehler in einer Verarbeitung brechen die übrigen Aktualisierungen nicht ab. Ein erneuter Lauf ist idempotent und erzeugt auf bereits aktuellen Branches keinen zusätzlichen Commit.

Falls das Runner-Kennzeichen bei der erstmaligen Einrichtung noch nicht im zentralen Workflowstand enthalten ist, übernimmt der Vorbereitungsjob es mit einem Commit und verwendet dessen SHA als Rollout-SHA.

1.5 5. Branches, Weitergabe und Auslöser

Jede aktive Releaselinie besitzt drei Branches, zum Beispiel:

```
R261/Entwicklung
R261/Abnahme
R261/Bereitstellung
```

Damit ist aus jedem Branchnamen eindeutig erkennbar, zu welcher Releaselinie und Stage er gehört. Ein gemeinsamer Branch je Stage mit mehreren Releaseverzeichnissen, wie er in SVN verwendet wurde, wird nicht fortgeführt.

Für größere oder gemeinsam bearbeitete Änderungen kann ein Feature-Branch verwendet werden. Er löst keine Verteilung aus. Kleine Änderungen dürfen auch direkt auf dem Entwicklungsbranch entstehen. Ein Feature-Branch ist keine zusätzliche Stage des Freigabeprozesses.

Ein Push nach `Rnnn/Entwicklung` oder `Rnnn/Abnahme` startet automatisch die M/Text-Verteilung für die entsprechende Stage. Beim Übergang zur nächsten Stage wird eine fachlich ausgewählte Änderung per Cherry-Pick übernommen. Der Cherry-Pick erzeugt auf dem Zielbranch einen neuen Commit mit einer neuen SHA. Weitergegeben wird dieselbe Änderung, nicht derselbe Commit. Ob die Releaselinie fachlich eingerichtet ist, entscheidet die zentrale Releaselinienzuordnung.

Ein Push nach `Rnnn/Bereitstellung` erzeugt noch keine Lieferung. Erst ein Tag im Format `Rnnn.nnn` startet den Paketbau. Dabei wird geprüft, ob der Tag zur angegebenen Releaselinie gehört und vom Bereitstellungsbranch erreichbar ist. Der Tag wird als Git-Tag angelegt und einzeln gepusht. Ein GitHub Release wird nicht erzeugt. Wurde ein Tag irrtümlich angelegt, wird der dadurch gestartete Workflow-Lauf vor dem Löschen oder Korrigieren des Tags abgebrochen.

Manuelle Wiederholungen eines Workflows sind möglich, müssen aber denselben Commit verwenden. Vor einer erneuten M/Text-Verteilung prüft der Workflow, ob dieser Commit zum ausgewählten Branch gehört.

1.5.1 Neue Releaselinie einrichten

Eine neue Linie erhält drei Branches, je einen für Entwicklung, Abnahme und Bereitstellung, sowie einen Eintrag in `config/releaselinien.json`. Der Eintrag enthält die fachliche Releaselinie, die technische ETAPS-Linie und den Namen eines in `.github/config.json` vorhandenen Hostprofils. Die JCL-Werte stammen aus der Mandantenkonfiguration und dem zugeordneten Hostprofil. Die Zuordnung wird rollierend gepflegt: Beim Aufnehmen einer neuen Releaselinie wird die ausgeschiedene Zuordnung entfernt, sodass immer drei aktive Releaselinien enthalten sind.

Ausgangspunkt der neuen Branches ist normalerweise der letzte Release-Tag der bisherigen Linie. Dessen vollständiger Projektstand wird über den manuellen Sync-Workflow einmal nach Entwicklung und einmal nach Abnahme übertragen und anschließend in M/Text fachlich geprüft.

1.6 6. Workflows, Trigger und Abhängigkeiten

Die Mandanten-Repositories enthalten nur die fachlichen Auslöser. Sie rufen fest gepinnte wiederverwendbare Workflows aus `mtext-actions` auf. Die eigentliche Fachlogik liegt in Python.

Ein Ereignis (Trigger) in einem Mandanten-Repository startete dessen Trigger-Workflow, über den GitHub die fest gepinnte Workflowdatei aus `mtext-actions` lädt und ihre Jobs auf dem Runner des Mandanten-Workflows (lokal) ausführt. Der Runner checkt den festgelegten Commit des Mandanten-Repositories nach `source/` und `mtext-actions@automation_ref` nach

`automation/` aus und führt anschließend den Python-Code aus (siehe `workflows/README.md` in den Mandanten-Repos für weitere Informationen).

1.6.1 Gesamtzusammenhang

Prozessschritt	Auslöser	Trigger-Workflow	Zentraler Workflow	Python-Skript	Ergebnis
Workflowdateien einrichten oder <code>mtext-actions</code> -Version ausrollen	Manueller Batchstart in <code>mtext-actions</code> mit freigegebener vollständiger SHA	keiner	<code>update-mandant-workflows.yml</code>	<code>python -m workflow_configuration</code>	Zentrale Runnerwerte und alle Mandanten-Pins auf dieselbe Rollout-SHA geprüft und festgeschrieben
Mandantenkonfiguration prüfen	Push mit Änderung an <code>.github/config.json</code> auf einen Branch	<code>validate-config.yml</code>	<code>reusable-validate-config.yml</code>	<code>validate-config</code>	Konfiguration geprüft
Entwicklung synchronisieren	Push nach <code>Rnnn/</code> Entwicklung oder manueller Start	<code>sync-resources.yml</code>	<code>reusable-sync-resources.yml</code>	<code>sync-resources</code>	Vollständiger Projektstand des Ziel-Commits nach M/Text-Entwicklung synchronisiert
Abnahme synchronisieren	Push eines per Cherry-Pick übernommenen Commits nach <code>Rnnn/</code> Abnahme oder manueller Start	<code>sync-resources.yml</code>	<code>reusable-sync-resources.yml</code>	<code>sync-resources</code>	Vollständiger Projektstand des Ziel-Commits nach M/Text-Abnahme synchronisiert
Bereitstellungsstand fortschreiben	Cherry-Pick und Push nach <code>Rnnn/</code> Bereitstellung	keiner	keiner	keines	Nur Git-Branch fortgeschrieben. Noch keine Lieferung

Prozessschritt	Auslöser	Trigger-Workflow	Zentraler Workflow	Python-Skript	Ergebnis
Release bauen und übergeben	Push eines Tags Rnnn.nnn o der manueller Start mit vorhandenem Tag	<code>release.yml</code>	<code>reusable-release.yml</code>	<code>build-release,</code> <code>danach publish-mainframe</code>	FULL/DELTA gebaut, geprüft und per FTP/JES übergeben
<code>mtext-actions</code> testen	Pull Request in <code>mtext-actions</code> oder Push auf dessen <code>main</code>	entfällt	<code>ci.yml</code>	<code>unittest discover</code>	Zentrale Testfälle und Workflowverträge geprüft

1.6.2 Mandantenseitige Workflows

Datei	Trigger	Aufgabe
<code>validate-config.yml</code>	Push mit Änderung an <code>.github/config.json</code>	Prüfung der Mandantenkonfiguration starten
<code>sync-resources.yml</code>	Push nach Entwicklung oder Abnahme sowie manueller Start	Ressourcensynchronisation für die Zielumgebung starten
<code>release.yml</code>	Release-Tag sowie manueller Start	Releaseverarbeitung starten

1.6.3 Zentrale Workflows

Datei	Trigger	Aufgabe
<code>update-mandant-workflows.yml</code>	Manueller Start	Alle Mandantenbranches auf den gewünschten <code>mtext-actions</code> -Commit aktualisieren
<code>reusable-validate-config.yml</code>	Aufruf durch <code>validate-config.yml</code>	Mandantenkonfiguration prüfen

Datei	Trigger	Aufgabe
<code>reusable-sync-resources.yml</code>	Aufruf durch <code>sync-resources.yml</code>	M/Text-Server synchronisieren
<code>reusable-release.yml</code>	Aufruf durch <code>release.yml</code>	Releasepakete erstellen und an den Mainframe übergeben
<code>ci.yml</code>	Pull Request oder Push auf <code>main</code> in <code>mtext-actions</code>	Tests in <code>mtext-actions</code> ausführen

1.7 7. Konfigurationsdateien

1.7.1 config.json

Die Datei `.github/config.json` in den Mandanten-Repositories ist ein versionierter Bestandteil des Lieferstands und enthält einen `mandant` -Block mit folgenden Feldern:

Feld	Bedeutung
<code>kuerzel</code>	Bekanntes Mandantenkürzel für Paketnamen und Fragmentprojekte
<code>ispw</code>	CodePipeline-Instanz <code>T</code> (Test) oder <code>P</code> (Produktion, default)
<code>excluded_projects</code>	Optionale Liste sichtbarer Projektverzeichnisse, die weder synchronisiert noch paketiert werden
<code>hostprofile</code>	Ein oder mehrere frei benannte Hostprofile mit <code>assignment</code> und <code>stage</code>

1.7.1.1 Beispielhafte .github/config.json

```
{
  "mandant": {
    "kuerzel": "FI",
    "ispw": "P",
    "excluded_projects": [
      "LOMS_Testdaten"
    ],
  },
}
```

```

"hostprofile": {
  "FKT": {
    "assignment": "LOMS000066",
    "stage": "FKTE"
  },
  "JUR": {
    "assignment": "LOMS000067",
    "stage": "JURP"
  }
}
}
}

```

1.7.2 mandanten.json

Die folgende Tabelle listet die aktuell bekannten Projekte die in GitHub versioniert und paketierrt werden. Weitere Projekte können aufgenommen werden, ohne dass dafür Workflows oder Konfigurationen geändert werden müssen. Eine Abweichung von diesem Soll-Stand wird aber mit Warnungen im Workflow-Log gekennzeichnet. Die Zuordnung von Mandant zu Repository wird in `mtext-actions` unter `config/mandanten.json` zentral gepflegt.

Mandanten-Kürzel	Repository	Projekte
FI	<code><oms_team>/mtext-fi</code>	<code>Configuration</code> , <code>Fonts</code> , <code>LOMS_Framework</code> , <code>LOMS_Basis</code> , <code>LOMS_PKA</code>
IT	<code><oms_team>/mtext-autonom</code>	<code>LOMS_Autonom</code>
BY	<code><oms_team>/mtext-by</code>	<code>LOMS_Basis[BY]</code> , <code>LOMS_Autonom[BY]</code>
LH	<code><oms_team>/mtext-lh</code>	<code>LOMS_Basis[LH]</code> , <code>LOMS_Autonom[LH]</code>
NW	<code><oms_team>/mtext-nw</code>	<code>LOMS_Basis[NW]</code> , <code>LOMS_Autonom[NW]</code>
OS	<code><oms_team>/mtext-os</code>	<code>LOMS_Basis[OS]</code> , <code>LOMS_Autonom[OS]</code>
SA	<code><oms_team>/mtext-sa</code>	<code>LOMS_Basis[SA]</code> , <code>LOMS_Autonom[SA]</code>

1.7.3 releaselinien.json

Die zentrale Datei `config/releaselinien.json` enthält rollierend die Zuordnung von drei aktiven fachlichen Releaselinien zur jeweiligen technischen ETAPS-Linie und dem zugehörigen Hostprofil. Ihr aktueller Inhalt ist:

```
{
  "R260": {"etaps_linie": "en03", "hostprofil": "JUR"},
  "R261": {"etaps_linie": "en01", "hostprofil": "FKT"},
  "R270": {"etaps_linie": "en02", "hostprofil": "JUR"}
}
```

Bei einer *Synchronisation* bestimmt die Releaselinie via `etaps_linie` das M/Text Ziel. Bei einer *Release-Lieferung* bestimmt sie zunächst den Namen des Hostprofils. `assignment` und `CodePipeline-stage` dieses Profils werden anschließend aus der Mandantekonfiguration des getaggtten Commits gelesen. So ergibt beispielsweise R261 für die FI über FKT die Werte LOMS0000066 und FKTE. Vor einer Verteilung oder Lieferung wird die gesamte benötigte Konfiguration geprüft. Unbekannte Mandanten, Releaselinien, Zielumgebungen oder zusätzliche Konfigurationsfelder führen zu einem Fehler.

1.8 8. Release-Lieferarten FULL und DELTA

Ein Tag mit der Endung `.100`, zum Beispiel `R261.100`, erzeugt für jedes sichtbare, nicht ausgeschlossene Projekt ein vollständiges F-Paket und ein zusätzliches leeres D-Paket. Das D-Paket enthält nur das leere Projektverzeichnis und eine leere Löschrliste. Das bisherige Jenkins-Skript erzeugt und übergibt bei FULL ebenfalls beide Pakete.

Jeder andere gültige Release-Tag derselben Releaselinie erzeugt ein kumulatives DELTA gegen den `.100`-Tag. Ein Tag `R261.108` enthält somit alle neuen, geänderten und gelöschten Dateien seit `R261.100`. Frühere DELTA-Lieferungen müssen nicht lückenlos eingespielt worden sein.

Die `.100`-Basis muss in der Git-Historie ein Vorgänger des Ziel-Tags sein. Git bestimmt die geänderten, neuen, gelöschten und umbenannten Pfade mit `git diff`. Python erzeugt daraus das historisch kompatible TAR-Archiv, die Löschrliste und die Informationsdatei mit reproduzierbaren Dateimetadaten.

1.8.1 CodePipeline-Elemente

Bei einer DELTA-Lieferung entsteht für jedes ausgelieferte Projekt ein D-Element. Bei einer FULL-Lieferung entstehen ein F-Element mit dem vollständigen Projektstand und zusätzlich ein leeres D-Element. Der Elementname ist zugleich der Mainframe-Member und setzt sich aus Mandantenkürzel, abgeleitetem Projektcode und Elementart zusammen.

```
<Mandantenkürzel><Projektcode><F|D>
```

Beispielsweise bezeichnet `BYAUTOND` das DELTA-Element für `LOMS_Autonom[BY]`. Eine FULL-Lieferung von `LOMS_Basis` der FI erzeugt `FIBASISF` mit dem vollständigen Projektstand sowie ein leeres `FIBASISD`.

Projekt	"Projektcode"
Configuration	CONFI
Fonts	FONTS
LOMS_Framework	FRAME
LOMS_Basis	BASIS
LOMS_PKA	PKA
LOMS_Autonom	AUTON

Der "Projektcode" entsteht, indem ein vorhandenes Mandantensuffix und das Präfix `LOMS_` entfernt und anschließend höchstens die ersten fünf Zeichen in Großschreibung verwendet werden. Zwei Projekte desselben Repositorys dürfen nicht denselben Projektcode ergeben. Ein F-Element enthält den vollständigen Projektbaum. Ein reguläres D-Element enthält die kumulativ seit `.100` neuen und geänderten Dateien sowie die Löschliste. Das beim FULL zusätzlich erzeugte D-Element enthält ein leeres Projektverzeichnis und eine leere Löschliste.

Releaseartefakt und Manifest

1.8.2 Releaseartefakt und Manifest

Für jeden Release-Lauf speichert GitHub Actions die erzeugten Pakete zusammen mit Manifest und Prüfsummen als Releaseartefakt. Der Publish-Job verwendet genau dieses Artefakt für die Mainframe-Übergabe. Das bisherige feste Verzeichnis `trans` wird für diesen Zweck nicht mehr benötigt. Dateinamen, fachlicher Inhalt und Archivstruktur der Pakete bleiben unverändert.

Ein DELTA-Artefakt für ein Projekt der FI enthält beispielsweise:

```
Release-Artefakt/
FIBASISD.tgz
_INFO_FI-LOMS_Basis-DELTA-R261.108-R261.107.txt
manifest.json
```

Ein FULL-Artefakt enthält für dasselbe Projekt beide Mainframe-Pakete:

```

Release-Artefakt/
  FIBASISF.tgz
  FIBASISD.tgz
  _INFO_FI-LOMS_Basis-FULL-R261.100-<Vorrelease>.txt
  manifest.json

```

Zu jeder Lieferung wird ein Manifest erzeugt. Die Begleitdatei nennt Repository, Mandant, Release-Tag, Lieferart, Basis- und Vorgänger-Tag, Ziel-SHA, JCL-Werte sowie alle Paket- und Informationsdateien mit Pfad, Größe und SHA-256. Paketartefakte nennen zusätzlich ihren Mainframe-Member. Vor der Mainframe-Übergabe werden genau diese Dateien mit den manifestierten Angaben verglichen. So wird sichergestellt, dass das zuvor gebaute und freigegebene Paket übergeben wird.

Der folgende gekürzte Ausschnitt zeigt die Verbindung zwischen Release, Paket, Mainframe-Member und JCL-Werten. Dateigröße und Prüfsummen stehen stellvertretend für die im jeweiligen Lauf berechneten Werte:

```

{
  "artifacts": [
    {
      "kind": "package",
      "member": "FIBASISD",
      "path": "FIBASISD.tgz",
      "project": "LOMS_Basis",
      "sha256": "<SHA-256 des Pakets>",
      "size": 123456
    },
    {
      "kind": "information",
      "path": "_INFO_FI-LOMS_Basis-DELTA-R261.108-R261.107.txt",
      "project": "LOMS_Basis",
      "sha256": "<SHA-256 der Informationsdatei>",
      "size": 1234
    }
  ],
  "base_tag": "R261.100",
  "delivery_type": "DELTA",
  "jcl": {
    "ASSIGNMENT": "LOMS0000066",
    "ISPW": "P",
    "LEVEL": "FKTE",
    "SUBSYS": "LOMS"
  }
}

```

```

},
"mandant": "FI",
"previous_tag": "R261.107",
"release_tag": "R261.108",
"repository": "mtext-fi",
"target_sha": "<vollständige Commit-SHA>"
}

```

Ein fehlgeschlagener Übergabeversuch kann innerhalb desselben GitHub-Laufs mit dem unveränderten Paket wiederholt werden (das Paket wird dabei nicht neu gebaut).

1.9 9. Mainframe-Übergabe und JCL

Die JCL liegt als eigene versionierte Template-Datei vor. Änderungen an der Mainframe-Ansteuerung sind dadurch sichtbar und unabhängig vom Programmcode prüfbar.

Für jede Übergabe werden ausschließlich die fachlich festgelegten Werte in das Template eingesetzt. Historisch feste Werte bleiben fest. Nur tatsächlich mandantenspezifische Zuordnungen werden aus der Mandantenkonfiguration übernommen. Die Werte für ISPW, CodePipeline-Stage, Assignment und Mainframe-Member werden beim Rendern geprüft. Unbekannte Template-Marker führen vor der Übergabe zu einem Fehler. Zugangsdaten werden weder in die JCL noch in die Protokolle geschrieben.

Das Paket wird zunächst unter seinem Membernamen in `IEA.LOMS.TONICZ` übertragen. Die JCL kopiert diesen Member nach `IEA.ISPW<ISPW>.BOAS.<LEVEL>.TONICZ` und registriert ihn anschließend in CodePipeline. Dabei gelten `STRMNAME=BOAS`, `MTYPE=TONICZ` und `MNAME=<Membername>`. `APPLID` und `SUBAPPL` erhalten das aus dem Mandantenkürzel abgeleitete Subsystem, `PROJNO` das Assignment sowie `CLVL` und `SLVL` den CodePipeline-Stage-Code.

Der Release-Workflow trennt den Paketbau von der Mainframe-Übergabe. Der Paketbau benötigt keinen Zugriff auf das Zielsystem. Erst der Übergabeschritt erhält Zugriff auf die geschützte Umgebung `Bereitstellung`.

Übergaben desselben Mandanten werden nacheinander ausgeführt. Verschiedene Mandanten können gleichzeitig liefern.

1.10 10. Status und Fehler

Die Lösung meldet nur den Status, den sie selbst sicher feststellen kann:

Status	Bedeutung
CONFIG_VALIDATED	Mandantenkonfiguration und Releaselinienzuordnung wurden technisch geprüft.
VALIDATION_FAILED	Eingaben oder Konfiguration sind ungültig.
SOURCE_FAILED	Der angegebene Commit, Branch oder Tag konnte nicht eindeutig aufgelöst werden.
RESOURCE_TRANSFER_FAILED	Die Ressourcen konnten nicht in den Übergabebereich für M/Text geschrieben werden.
ADAPTER_FAILED	Der M/Text-Adapter war nicht erreichbar oder hat die Anfrage abgelehnt.
ADAPTER_ACCEPTED	Der M/Text-Adapter hat die Anfrage unmittelbar angenommen. Dies ist noch kein fachlicher Endstatus. Bei unklarer Wirkung ermittelt die Anwendungsbetreuung den technischen Anwendungsstatus.
PACKAGE_FAILED	Paket, Informationsdatei oder Manifest konnten nicht korrekt erstellt werden.
ARTIFACT_READY	Das Releasepaket wurde vollständig erstellt und geprüft.
MAINFRAME_TRANSFER_FAILED	Die unmittelbare FTP-/JES-Übergabe ist fehlgeschlagen.
MAINFRAME_SUBMITTED	Paket und JCL wurden technisch übergeben. Der spätere Mainframe-Job kann trotzdem noch fehlschlagen und wird durch das Mandanten-Release-Team auf dem Host kontrolliert.

Ein HTTP-Fehler des M/Text-Adapters gilt immer als fehlgeschlagener Lauf. Ein Status zwischen 200 und 299 bestätigt nur die unmittelbare Annahme der Anfrage.

Die Automatisierung fragt weder bei M/Text noch auf dem Mainframe nach dem späteren fachlichen Endstatus.

1.11 11. Qualitätsmerkmale und weitere Ausbaustufe

1.11.1 Tragende Qualitätsmerkmale

Gemessen an modernen Best Practices für GitHub Actions und automatisierte Softwarelieferungen besitzt die Lösung eine starke technische Grundlage. Die Einordnung orientiert sich insbesondere an den GitHub-Empfehlungen zu [wiederverwendbaren Workflows](#)¹, zum [sicheren Einsatz von GitHub Actions](#)² und zu [geschützten Environments](#)³.

Qualitätsmerkmal	Umsetzung und Nutzen
Durchgängiger Gesamtprozess	Die fachliche Kette führt von Entwicklung über Abnahme und Bereitstellung zum Release-Tag. Jeder Übergang hat einen eindeutigen Auslöser und ein prüfbares Ergebnis.
Zentral gepflegte Automatisierung	Die Mandanten-Workflows enthalten nur Trigger und feste Zielzuordnungen. Die gemeinsame Fachlogik liegt in wiederverwendbaren Workflows und einer Python-Implementierung (Standardbibliothek) in <code>mtext-actions</code> .
Eindeutige und reproduzierbare Lieferung	Jeder Lauf verarbeitet einen vollständigen Commit-SHA. Das Manifest verbindet Release-Tag, Ziel-Commit und erzeugte Dateien. Gleiche Eingaben erzeugen bytegleiche Archive. Historische Namen, Verzeichnisstrukturen, Löschlisten und JCL-Verträge bleiben erhalten.
Getrennte Verantwortlichkeiten	Mandantenressourcen und -konfiguration, gemeinsame Automatisierung, GitHub-Schutzregeln und Runnerbetrieb haben jeweils einen klaren Eigentümer.
Geprüfte Build-Publish-Grenze	Der Paketbau ist von der Mainframe-Übergabe getrennt. Das einmal erzeugte Artefakt wird unmittelbar vor der externen Wirkung anhand von Pfad, Größe und SHA-256 geprüft.
Automatisiert prüfbarer Vertrag	Tests decken Konfiguration, Git-Bezüge, FULL und DELTA, Manifest, JCL, Ressourcensynchronisation, FTP/JES und Workflowgrenzen ab. Stabile Statuswerte unterscheiden die Fehlerklassen.

1.11.2 Mögliche Phase 2

Die erste Ausbaustufe bleibt bewusst auf die sichere Ablösung des bestehenden Lieferwegs begrenzt. Die in [Nächste Schritte](#) (see page 47) geführten Einrichtungs- und Abnahmepunkte sind Voraussetzungen für die Aktivierung und keine Phase-2-Themen. Nach einem stabilen Produktivbetrieb können insbesondere folgende Erweiterungen bewertet werden:

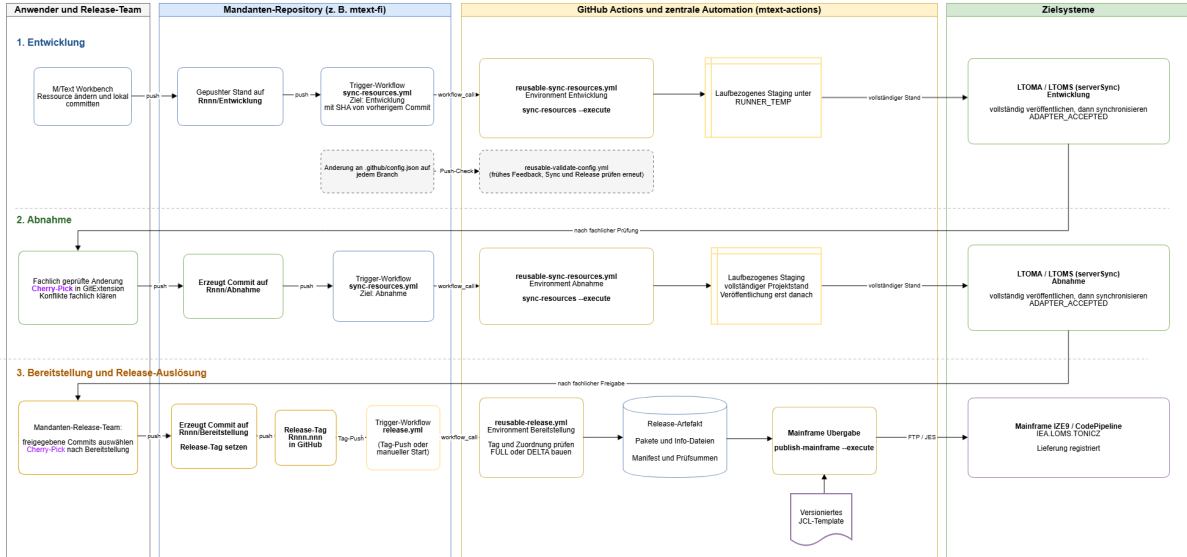
1. <https://docs.github.com/en/enterprise-server@3.20/actions/concepts/workflows-and-actions/reusing-workflow-configurations>
2. <https://docs.github.com/en/enterprise-server@3.20/actions/reference/security/secure-use>
3. <https://docs.github.com/en/enterprise-server@3.20/actions/reference/workflows-and-actions/deployments-and-environments>

- den nachgelagerten fachlichen Status in M/Text und auf dem Mainframe abfragen und im Workflow anzeigen (Polling),
- die FTP-/JES-Übergabe auf einen verschlüsselten Transport umstellen, sobald das Zielsystem dafür einen verbindlichen Vertrag bereitstellt,
- Betriebsmetriken und kompakte Laufzusammenfassungen ergänzen, ohne mandantenübergreifende oder vertrauliche Details offenzulegen,
- zusätzliche E-Mail-Benachrichtigungen für relevante Workflow-Ergebnisse ergänzen, ohne den fachlichen Laufstatus von der Benachrichtigung abhängig zu machen,
- Aktualisierungen gepinnter Actions sowie ergänzende Workflow-, Shell-, Typ- und Abdeckungsprüfungen automatisieren.

2 Ablaufdiagramm Git (neu)

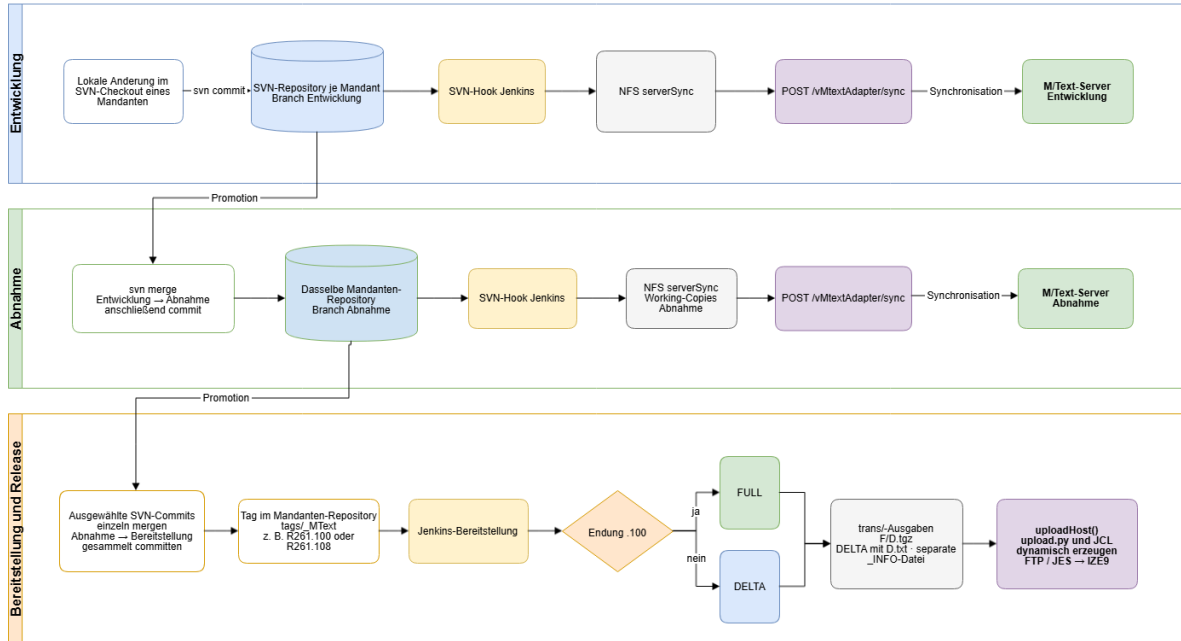
Soll-Ablauf: Git / GitHub Actions / M/Text / Mainframe

Mandanten-Repository → fest geprüfte zentrale Automation → Zielsysteme | Test-Parallelbetrieb Nov./Dez. 2026, Cutover ab Jan. 2027



3 Ablaufdiagramm SVN (alt)

Ist-Ablauf: SVN / Jenkins / M/Text / Mainframe



4 Benutzeranleitung

4.1 Benutzeranleitung für die M/Text-Lieferung mit GitHub Actions

4.1.1 1. Zweck

Diese Anleitung begleitet Text-Entwickler und Administratoren durch den vollständigen Bedienablauf. Alle Angaben, die sie für ihre jeweilige Aufgabe benötigen, stehen in diesem Dokument.

Die Lösung führt M/Text-Ressourcen über die drei Stages `Entwicklung`, `Abnahme` und `Bereitstellung`. Pushes nach `Entwicklung` und `Abnahme` lösen die Synchronisation mit dem jeweiligen M/Text-Server aus. Ein Push nach `Bereitstellung` liefert noch nichts aus. Erst ein Release-Tag startet eine FULL- oder DELTA-Lieferung mit anschließender Mainframe-Übergabe an die IZE9.

4.1.1.1 Abgrenzung

Die technische Ersteinrichtung und der Cutover gehören nicht zur normalen Bedienung. Für die dafür verantwortlichen Teams sind sie zusätzlich in [Nächste Schritte](#) (see page 47) beschrieben.

Die konkrete Benutzerverwaltung erfolgt je Mandanten-Repository. Für die `Bereitstellung` benennt jeder Mandant ein Mandanten-Release-Team mit wenigen berechtigten Personen.

4.1.2 2. Begriffe und Namensregeln

Eine **Stage** ist in dieser Anleitung ein Abschnitt des Freigabe- und Lieferprozesses - bei uns wie gewohnt `Entwicklung`, `Abnahme` und `Bereitstellung`.

Jede Releaselinie besitzt einen eigenen Branch für jede dieser Stages.

Element	Format	Beispiel
Branch einer Stage	<code>Rnnn/Entwicklung</code> , <code>Rnnn/Abnahme</code> , <code>Rnnn/Bereitstellung</code>	<code>R270/Entwicklung</code>
Optionaler Feature-Branch	<code>feature/Rnnn/<Issue>-<kurzname></code>	<code>feature/R270/0711-komplizierte-Arbeiten</code>
Release-Tag	<code>Rnnn.nnn</code>	<code>R261.100</code> , <code>R261.108</code>

Die Groß- und Kleinschreibung der Stage-Namen ist verbindlich. Die zentrale Konfiguration enthält derzeit die Releaselinien `R260`, `R261` und `R270`.

4.1.3 3. Grundlagen zu Git

4.1.3.1 Unterschied zwischen SVN und Git einfach erklärt

SVN und Git erfüllen denselben Grundzweck: Sie speichern Änderungen an Dateien und machen frühere Stände nachvollziehbar. Der wichtigste Unterschied liegt darin, wie Änderungen gespeichert und mit anderen geteilt werden.

SVN	Git
Die gemeinsame Versionsverwaltung liegt zentral auf dem SVN-Server.	Jeder ausgecheckte Arbeitsbereich ist ein eigenes lokales Git-Repository mit Versionshistorie. GitHub dient als gemeinsame, zentrale Austauschplattform.
Ein Commit überträgt eine Änderung direkt an den SVN-Server.	Ein Commit speichert die Änderung zunächst nur lokal. Erst ein anschließender Push überträgt sie nach GitHub.
Neue Stände erhalten fortlaufende Revisionsnummern wie <code>r12345</code> .	Jeder Stand erhält eine eindeutige Commit-SHA.
Branches werden vergleichsweise selten und eher für länger getrennte Entwicklungslinien verwendet.	Branches lassen sich leicht als vorübergehende Arbeitsbereiche für einzelne Änderungen nutzen.

Für die tägliche Arbeit bedeutet das unter anderem: **Committen und Pushen sind in Git zwei getrennte Schritte**. Ein lokaler Commit sichert den eigenen Arbeitsstand, verändert aber noch nichts auf GitHub und löst keine Synchronisation aus. Erst der Push auf den Branch einer Stage veröffentlicht den Stand und startet – bei Entwicklung oder Abnahme – den zugehörigen Workflow.

Git und GitHub sind dabei nicht dasselbe: **Git** ist die Versionsverwaltung. **GitHub** stellt das gemeinsame Repository, die Oberfläche, Berechtigungen und die GitHub-Actions-Automation bereit.

4.1.3.2 Commit und SHA einfach erklärt

Ein **Commit** ist ein gespeicherter Stand im Git-Repository. Jeder Commit hat eine eindeutige technische Kennung, den **Commit-SHA**. Sie besteht aus 40 Zeichen, zum

Beispiel `8f3a1c2d4e5f67890123456789abcdef01234567`. (Meist genügt in der Oberfläche eine verkürzte Darstellung. Für einen manuellen Wiederanlauf verlangt die Automation jedoch die vollständige Kennung.)

Ein Branch-Name wie `R261/Entwicklung` bezeichnet dagegen keinen dauerhaft festen Stand. Er zeigt auf den jeweils neuesten Commit des Branches und bewegt sich mit jedem weiteren Push!

4.1.3.3 SVN-Pfad und Git-Branch unterscheiden

Im bisherigen SVN-Aufbau sind Stage und Releaselinie Bestandteile des Pfads. Bei der Migration werden diese Informationen auf Git-Branches abgebildet:

SVN: `branches/Entwicklung/R260.100_MText/<Projekt>`

Git: Branch `R260/Entwicklung`, darin Pfad `<Projekt>`

Releaselinie und Stage stehen im Branch-Namen. Im Arbeitsbereich beginnt der Pfad direkt mit dem Projektverzeichnis.

4.1.3.4 Lokales Repository vor der Arbeit aktualisieren

Vor einer Bearbeitung, einem Cherry-Pick oder dem Anlegen eines Release-Tags muss der lokale Branch auf dem aktuellen GitHub-Stand sein. In der M/Text Workbench wird dafür das Eclipse-Plugin **EGit** verwendet.

Git unterscheidet das Abrufen neuer GitHub-Stände vom Aktualisieren des ausgecheckten Branches:

Git-Vorgang	Wirkung auf den lokalen Stand
<code>fetch</code>	Lädt neue Commits und Branchstände von GitHub, verändert aber weder den ausgecheckten lokalen Branch noch dessen Dateien
<code>pull</code>	Ruft den GitHub-Stand ab und aktualisiert den ausgecheckten Branch

Welche EGit-Bedienfunktion diese Vorgänge in der M/Text Workbench und im zusätzlichen Git-Client ausführt und wie sie dort heißt, wird erst bei der praktischen Abnahme festgelegt. Die freigegebene Funktion muss den lokalen Branch ohne automatischen Merge auf den aktuellen GitHub-Stand bringen.

Der Standardablauf in der Workbench ist:

1. Das richtige Mandanten-Repository und den vorgesehenen Branch auswählen.
2. Prüfen, dass keine noch nicht abgeschlossene Git-Operation und keine ungesicherte lokale Bearbeitung vorliegt.
3. Den lokalen Branch mit der dafür abgenommenen Bedienfunktion auf den aktuellen GitHub-Stand bringen. Ein reiner `fetch` genügt dafür noch nicht.
4. Kontrollieren, dass lokaler Branch und GitHub-Branch anschließend auf denselben Commit zeigen.
5. Erst danach mit Bearbeitung, Cherry-Pick oder Tag-Erzeugung beginnen.

Lehnt die Aktualisierung wegen eigener lokaler Commits ab, wird weder ein Merge noch ein Force-Push erzwungen. Dann gilt der Abschnitt [Push-Ablehnung mit Rebase statt Merge beheben](#) (see page 30).

4.1.3.5 Feature-Branch einfach erklärt

Ein **Feature-Branch** ist ein eigener Arbeitsbereich für eine einzelne Änderung. Damit kann ein noch unfertiger Stand gespeichert und nach GitHub gepusht werden, ohne bereits das M/Text-Entwicklungssystem zu aktualisieren. Das ist besonders nützlich, wenn eine Änderung mehrere Tage dauert oder mehrere Personen daran arbeiten.

Für eine kleine, fertig vorbereitete Änderung ist kein Feature-Branch nötig. Sie kann direkt auf dem passenden Entwicklungsbranch bearbeitet und gepusht werden. Ist die Arbeit auf einem Feature-Branch abgeschlossen, wird der gewünschte Commit nach `<ReleaseLine>/`

Entwicklung übernommen. Erst der Push dieses Entwicklungsbranches startet die M/Text-Synchronisation. Der Feature-Branch selbst bezeichnet keine Stage des Lieferprozesses und löst weder eine M/Text-Synchronisation noch einen Release aus. Nur wenn `.github/config.json` geändert wird, läuft dort der Config-Check.

4.1.3.6 Cherry-Pick einfach erklärt

Ein **Cherry-Pick** übernimmt die Änderung eines gezielt ausgewählten Commits auf einen anderen Branch. Anders als bei der Zusammenführung eines ganzen Branches werden dadurch nicht automatisch alle dort vorhandenen Änderungen weitergegeben. Das ist für den vorgesehenen Ablauf zwischen den Stages wichtig: Nach Abnahme und Bereitstellung sollen nur fachlich ausgewählte Änderungen übernommen werden.

Der Cherry-Pick erzeugt auf dem Zielbranch einen **neuen Commit mit einer neuen SHA**. Es wird also dieselbe Änderung weitergegeben, nicht derselbe Commit.

Mehrere freigegebene Commits können in ihrer ursprünglichen Reihenfolge nacheinander per Cherry-Pick übernommen und anschließend gemeinsam mit einem einzigen Push übertragen werden. Jeder dabei neu erzeugte Commit dokumentiert die vollständige SHA seines jeweiligen Quell-Commits.

Das folgende Beispiel zeigt die selektive Weitergabe. Auf Entwicklung liegen die unabhängigen Änderungen A, B und C. Nach Abnahme soll zunächst nur B:

Entwicklung:	Basis – A – B – C
Abnahme:	Basis – B'

`B'` enthält dieselbe fachliche Änderung wie B, ist aber wegen des anderen Eltern-Commits und der neuen Metadaten ein eigener Commit mit neuer SHA. A und C werden nicht automatisch übernommen. Beim nächsten Übergang wird der in Abnahme geprüfte Commit `B'` weitergegeben:

Entwicklung:	Basis – A – B – C
Abnahme:	Basis – B'
Bereitstellung:	Basis – B'

4.1.3.7 Push-Ablehnung mit Rebase statt Merge beheben

Wenn zwei Personen vom selben GitHub-Stand aus arbeiten und eine Person zuerst pusht, baut der lokale Commit der zweiten Person noch auf dem vorherigen Stand auf. Git lehnt deren Push deshalb üblicherweise als `non-fast-forward` ab. Das kann auch geschehen, wenn beide Personen unterschiedliche Dateien geändert haben.

Diese Ablehnung ist noch kein inhaltlicher Konflikt. Sie bedeutet zunächst nur, dass der lokale Branch nicht mehr aktuell ist. Der neue GitHub-Stand muss zuerst übernommen werden. Sind die Änderungen unabhängig, kann Git sie automatisch kombinieren. Überschneiden oder widersprechen sie sich, entsteht dabei ein Konflikt.

Ein **Rebase** ist dann hilfreich, wenn der GitHub-Branch inzwischen fortgeschritten ist und zusätzlich eigene, noch nicht gepushte Commits vorliegen. Nach `fetch` setzt der Rebase diese eigenen Commits auf den aktuellen Remote-Branch. Überschneidungen werden dabei Commit für Commit aufgelöst. Ein Rebase ist ausschließlich für eigene, noch nicht veröffentlichte Commits zulässig, weil er neue Commit-SHAs erzeugt.

Ein **Merge** verbindet zwei Entwicklungsverläufe, ohne vorhandene Commits umzuschreiben, und kann dabei einen zusätzlichen Merge-Commit erzeugen. Er ist in diesem Prozess weder zur Behebung einer Push-Ablehnung noch zur Weitergabe zwischen den Stage-Branches vorgesehen. Eigene noch nicht gepushte Commits werden per Rebase auf den aktuellen Stand gesetzt. Freigegebene Änderungen werden per Cherry-Pick in die nächste Stage übernommen.

Situation	Geeignetes Vorgehen	Ergebnis
Push wird abgelehnt, weil GitHub fortgeschritten ist, und die eigenen Commits sind noch nicht gepusht	<code>fetch</code> , danach <code>rebase</code> auf den Remote-Branch	Eigene Commits werden auf dem aktuellen GitHub-Stand neu erzeugt
Cherry-Pick in die nächste Stage meldet einen Konflikt	Konflikt innerhalb des laufenden Cherry-Picks auflösen oder den Cherry-Pick vollständig abbrechen	Nur die ausgewählte Änderung wird in den Zielbranch übernommen
Fehlerhafter Commit ist bereits gepusht	Gegen-Commit nach dem beschriebenen Rücknahmeverfahren	Gemeinsame Historie bleibt unverändert nachvollziehbar

4.1.3.8 Konflikt bei Rebase oder Cherry-Pick auflösen

Ein Konflikt bedeutet, dass Git den gewünschten Dateiinhalt nicht eindeutig bestimmen kann. Er wird im freigegebenen Git-Client wie folgt bearbeitet:

1. Nicht pushen und zunächst prüfen, ob gerade ein Rebase oder ein Cherry-Pick läuft und welche Dateien betroffen sind.

2. Für jede Konfliktstelle den bisherigen Inhalt des Zielbranches, die zu übernehmende Änderung und den gemeinsamen Ausgangsstand vergleichen. Nicht pauschal eine komplette Dateiversion auswählen, wenn dadurch fachliche Änderungen der anderen Seite verloren gehen.
3. Den fachlich richtigen Zielinhalt herstellen, alle Konfliktmarkierungen entfernen und die aufgelösten Dateien zur Fortsetzung auswählen.
4. Die laufende Operation fortsetzen. Treten weitere Konflikte auf, werden dieselben Schritte für den nächsten Commit wiederholt.
5. Nach Abschluss Arbeitsstand, Änderungen und Commit-Historie prüfen. Erst danach darf der Branch ohne Force-Push übertragen werden.

Operation	Nach Auflösung fortsetzen	Vollständig abbrechen
Rebase	Dateien mit <code>add</code> als aufgelöst markieren, danach <code>rebase --continue</code>	<code>rebase --abort</code>
Cherry-Pick	Dateien mit <code>add</code> als aufgelöst markieren, danach <code>cherry-pick --continue</code>	<code>cherry-pick --abort</code>

Ist die fachlich richtige Auflösung nicht eindeutig, wird die Operation abgebrochen und die Abweichung mit den Verantwortlichen der betroffenen Änderungen geklärt. Die normale Aktualisierungsfunktion benötigt keinen solchen Auflösungsablauf: Kann sie den lokalen Branch nicht direkt auf den GitHub-Stand setzen, bricht sie ohne automatischen Merge ab.

4.1.4 4. Git-Anwendungen bedienen

Für den Gesamtprozess werden zwei Anwendungen mit klar getrennten Aufgaben verwendet:

Anwendung	Aufgabe	Benutzerkreis
M/Text Workbench mit integriertem Git-Client (Egit Plugin)	Briefressourcen bearbeiten, Änderungen prüfen, committen und nach Entwicklung pushen	Text-Entwickler
GitHub im Browser	Commits, Release-Tags und Läufe prüfen sowie manuelle Läufe starten	Text-Entwickler sowie die jeweils berechtigten Verantwortlichen

Hilfreiche Git-Funktionen

4.1.4.1 Benötigte Git-Funktionen

Anwendungsfall	Git-Funktion beziehungsweise Befehlsentsprechung	Nächstliegende SVN-Funktion
Arbeitsstand und ausgewählte Änderungen prüfen	<code>status</code> und <code>diff</code>	<code>svn status</code> und <code>svn diff</code>
GitHub-Stand abrufen und lokalen Branch ohne zusätzlichen Merge aktualisieren	<code>fetch</code> und anschließend <code>pull</code> beziehungsweise die freigegebene Aktualisierungsfunktion des Clients	<code>svn update</code>
Stage-Branch auswählen	<code>switch</code> beziehungsweise die Branchauswahl des Clients	<code>svn switch <URL></code>
Dateien auswählen, Commit erzeugen und übertragen	<code>add</code> , <code>commit</code> und <code>push</code>	<code>svn add</code> und <code>svn commit</code> . Der Commit überträgt die Änderung zugleich
Commit-Historie, vollständige SHA und konkrete Änderungen prüfen	<code>log</code> und <code>show</code>	<code>svn log</code> und <code>svn diff -c <Revision></code>
Letzten lokalen, noch nicht gepushten Commit ergänzen oder seine Nachricht korrigieren	<code>commit --amend</code> , nur vor dem Push, da dabei eine neue Commit-SHA entsteht	Keine direkte Entsprechung, weil ein SVN-Commit bereits veröffentlicht ist
Ausgewählten Commit in die nächste Stage übernehmen und Konflikte behandeln	<code>cherry-pick</code> , nach der geprüften Auflösung <code>cherry-pick --continue</code> oder zum vollständigen Abbruch <code>cherry-pick --abort</code>	<code>svn merge -c <Revision> <URL></code> . Konflikte werden vor dem anschließenden <code>svn commit</code> in der Arbeitskopie aufgelöst
Release-Tag auf einem bestätigten Commit anlegen, einzeln pushen und bei Bedarf wieder löschen	<code>tag</code> sowie gezielter Push oder Löschung der betreffenden Tag-Referenz. Die konkrete Bedienung wird mit dem zusätzlichen Git-Client abgenommen	<code>svn copy <Quell-URL> <Tag-URL></code> beziehungsweise <code>svn delete <Tag-URL></code> . Ein SVN-Tag ist ein Repositorypfad

Anwendungsfall	Git-Funktion beziehungsweise Befehlsentsprechung	Nächstliegende SVN-Funktion
Inzwischen fortgeschrittenen Remote-Branch mit eigenen, noch nicht gepushten Commits zusammenführen	<code>fetch</code> , danach kontrolliertes <code>rebase</code> auf den Remote-Branch. Konflikte auflösen und mit <code>rebase --continue</code> fortfahren oder mit <code>rebase --abort</code> abbrechen	<code>svn update</code> integriert lokale, noch nicht committete Änderungen. Lokale SVN-Commits gibt es nicht
Noch nicht committete Arbeit vor einem Branchwechsel vorübergehend beiseitelegen	<code>stash push</code> und später <code>stash pop</code> . Danach die wiederhergestellten Änderungen vollständig prüfen	Keine direkte, durchgängig verfügbare Standardentsprechung
Lokale Änderung, lokalen Commit oder gepushten Commit zurücknehmen	je nach Zustand <code>restore</code> , <code>reset</code> oder <code>revert</code> wie im Abschnitt Änderung oder Commit zurücknehmen (see page 34) beschrieben	Lokal <code>svn revert</code> . Eine veröffentlichte Änderung wird durch umgekehrten Merge und anschließenden <code>svn commit</code> korrigiert

Die Weitergabe zwischen den Stages erfolgt per Cherry-Pick und nicht durch einen Merge ganzer Branches. `stash` ist nur eine vorübergehende lokale Ablage und kein Ersatz für einen geprüften Commit. `clean` gehört nicht zum normalen Bedienweg, weil es nicht versionierte und damit möglicherweise noch nicht gesicherte Ressourcendateien unwiederbringlich entfernen kann. Release-Tags werden ausschließlich durch das Mandanten-Release-Team nach dem beschriebenen Verfahren in GitHub verwaltet.

Als herstellerseitige Vertiefung dienen die offiziellen Git-Anleitungen [Git-Grundlagen: Änderungen nachverfolgen und speichern](#)⁴ und [Ungewollte Änderungen rückgängig machen](#)⁵ sowie [Egit-Benutzerhandbuch \(see page 26\)](#) (https://help.eclipse.org/latest/topic/org.eclipse.egit.doc/help/EGit/User_Guide/User-Guide.html) für die Bedienung in der Eclipse-basierten M/Text Workbench.

4.1.4.2 Zusätzlicher Git-Client

Das Eclipse-Plugin **EGit** wird für die Ressourcenarbeit und die gezielte Weitergabe zwischen den Branches verwendet. Der Standardablauf ist:

1. Das Mandanten-Repository öffnen und die neuesten GitHub-Stände abrufen.

4. <https://git-scm.com/book/de/v2/Git-Grundlagen-%C3%84nderungen-nachverfolgen-und-im-Repository-speichern>

5. <https://git-scm.com/book/de/v2/Git-Grundlagen-Ungewollte-%C3%84nderungen-r%C3%BCckg%C3%A4ngig-machen.html>

2. Den Zielbranch auschecken und nach dem Abschnitt [Lokales Repository vor der Arbeit aktualisieren](#) (see page 28) auf den aktuellen GitHub-Stand bringen.
3. Die fachlich freigegebenen Quell-Commits auswählen und, falls Abhängigkeiten bestehen, ihre Reihenfolge festlegen.
4. Alle ausgewählten Commits einzeln und in der gewünschten Reihenfolge per Cherry-Pick übernehmen.
5. Bei einem Konflikt nicht pushen. Den Cherry-Pick entweder vollständig abbrechen oder nach dem Abschnitt [Konflikt bei Rebase oder Cherry-Pick auflösen](#) (see page 30) kontrolliert abschließen.
6. Geänderte Dateien und neu entstandene Ziel-Commits kontrollieren.
7. Die übernommenen Commits gemeinsam mit einem einzigen Push ohne Force-Push nach GitHub übertragen und dort den erreichten Ziel-Commit prüfen.

4.1.4.3 Änderung oder Commit zurücknehmen

Solange eine Änderung noch nicht gepusht wurde, kann sie lokal verworfen beziehungsweise ein lokaler Commit kontrolliert zurückgenommen werden. Es ist zu prüfen, dass dabei keine weiteren noch benötigten lokalen Änderungen verloren gehen.

Nach einem Push wird der Commit nicht aus dem Stage-Branch entfernt. Ein Force-Push oder das Zurücksetzen des gemeinsamen Branches ist verboten, weil die Commit-SHA bereits in GitHub-Läufen und gegebenenfalls in einer M/Text-Verteilung verwendet wurde. Stattdessen wird mit der Revert-Funktion in EGit ein neuer Commit erzeugt, der genau die Änderungen des fehlerhaften Commits umkehrt. Dieser Gegen-Commit wird wie jede andere Änderung geprüft und gepusht.

Die Git-Begriffe unterscheiden diese Fälle: `restore` verwirft noch nicht committete Änderungen an Dateien, `reset` nimmt einen noch nicht gepushten lokalen Commit zurück und `revert` erzeugt den Gegen-Commit für einen bereits gepushten Commit. Die konkreten Schaltflächen hängen vom freigegebenen Git-Client ab. Eine Git-Kommandozeile ist dafür nicht vorgeschrieben.

Ein `reset`, `rebase` oder `commit --amend` auf einem bereits gepushten Stage-Branch würden dessen gemeinsame Historie umschreiben und sind deshalb nicht zulässig.

Situation	Vorgehen
Lokale Änderung ohne Commit	Nur die nicht mehr benötigten Dateien oder Änderungen verwerfen und den verbleibenden Stand prüfen
Lokaler Commit ohne Push	Commit zurücknehmen. Anschließend lokalen Branch und geänderte Dateien prüfen
Commit auf Entwicklung oder Abnahme gepusht	Gegen-Commit auf demselben Branch erstellen und pushen. Der dadurch gestartete Sync verteilt den vollständigen korrigierten Stand
Commit nach Bereitstellung gepusht, aber noch nicht getaggt	Das Mandanten-Release-Team erstellt und pusht den Gegen-Commit. Ein Push nach Bereitstellung allein erzeugt keine Lieferung

Situation	Vorgehen
Commit bereits in eine weitere Stage übernommen	Den betroffenen Commit in jeder Stage, in die er übernommen wurde, mit den dort geltenden Berechtigungen durch einen eigenen Gegen-Commit zurücknehmen. Ein Gegen-Commit auf Entwicklung ändert Abnahme oder Bereitstellung nicht automatisch
Release-Tag falsch angelegt	Einen noch laufenden Ablauf abbrechen und den Tag gem. Verfahren in Kapitel 10 zurücknehmen. Den Bereitstellungsbranch bei Bedarf mit einem neuen Commit korrigieren und anschließend den korrekten Tag anlegen
Mainframe-Übergabe bereits erfolgt	Das Löschen des Tags nimmt die Lieferung nicht zurück. Eine fachliche Korrektur erfolgt mit einem neuen Commit und einem neuen Release-Tag nach dem regulären Verfahren

Ist der Commit fachlich richtig und nur der Workflow technisch fehlgeschlagen, wird kein Gegen-Commit erzeugt. In diesem Fall wird derselbe Commit nach der Fehlerbehebung kontrolliert wiederholt.

4.1.4.4 GitHub im Browser

GitHub im Browser dient der Kontrolle und den Prozessschritten, die nicht in der Workbench stattfinden.

4.1.4.4.1 Commit und Workflow-Lauf prüfen

1. Das Mandanten-Repository und dort **Actions** öffnen.
2. Den zum Prozessschritt gehörenden Workflow und den ausgelösten Lauf auswählen.
3. Branch oder Tag, Commit-SHA und Auslöser mit dem erwarteten Stand vergleichen.
4. Die Jobs und ihre Protokolle öffnen und den abschließenden Status prüfen.
5. Bei einem Fehler zuerst die früheste fachliche Fehlermeldung auswerten und keine Zugangsdaten oder andere vertrauliche Werte in Kommentare übernehmen.

4.1.4.4.2 Lauf abbrechen, wiederholen oder manuell starten

- Ein offensichtlich falscher Lauf wird vor einer externen Wirkung abgebrochen.
- **Re-run jobs** wiederholt einen vorhandenen Lauf mit demselben Commit und derselben Git-Referenz.
- **Run workflow** startet einen neuen kontrollierten Lauf mit den in Kapitel 11 beschriebenen Eingaben.
- Vor jeder Wiederholung wird geprüft, ob das Zielsystem den vorherigen Lauf bereits angenommen hat.

4.1.4.4.3 Mainframe-Übergabe freigeben

1. Im Release-Lauf prüfen, dass Tag, Ziel-Commit und Paketbau zum vorgesehenen Stand gehören.
2. Den automatisch gestarteten Publish-Job bis zum Abschluss kontrollieren.
3. Anschließend den Status auf dem Host kontrollieren.

Für diese Freigabe ist kein verpflichtendes Vier-Augen-Prinzip vorgesehen. Eine dafür berechnigte Person darf sowohl den Tag anlegen als auch den Publish-Job freigeben. Die inhaltlichen Prüfungen vor der Freigabe bleiben vollständig erforderlich.

Das Anlegen und die kontrollierte Rücknahme eines Release-Tags sind in Kapitel 10 beschrieben. Änderungen an Repositoryrechten, Rulesets, Environments oder Secrets gehören nicht zur normalen Benutzerarbeit. Einrichtung und Abnahme sind in [Nächste Schritte \(see page 26\)](#) beschrieben.

4.1.5 5. Mandantenkonfiguration in `.github/config.json`

Die Datei `.github/config.json` liegt im Verzeichnis `.github` des Mandanten-Repositorys und gehört wie die M/Text-Ressourcen zum versionierten Lieferstand. Sie beantwortet drei Fragen:

1. Zu welchem Mandanten gehört der Stand?
2. Welche Projektverzeichnisse werden ausdrücklich ausgeschlossen?
3. Welche mandantenspezifischen CodePipeline-Zuordnungen gelten für die nachgelagerte Übergabe?

4.1.5.1 Projektverzeichnisse und Ausschlüsse

Jedes sichtbare Verzeichnis direkt unter der Repositorywurzel ist ein Lieferprojekt. Es wird bei jedem passenden Entwicklung- oder Abnahmelaufl synchronisiert und bei einem Release als FULL- oder DELTA-Paket gebaut. Dateien oder Verzeichnisse, deren Name mit einem Punkt beginnt, werden automatisch ignoriert. Deshalb wird beispielsweise `.github` nicht als Projekt behandelt.

Die optionale Liste `excluded_projects` enthält ausschließlich Verzeichnisse, die trotz ihrer Lage in der Repositorywurzel nicht verarbeitet werden sollen. Das Beispiel der FI schließt die Testdaten aus:

```
"excluded_projects": [
  "LOMS_Testdaten"
]
```

Die technischen Namen der daraus erzeugten Paketdateien und Mainframe-Member werden aus dem Projektnamen abgeleitet. Dafür entfallen das Präfix `LOMS_` und ein Mandantensuffix in eckigen Klammern. Von dem verbleibenden Namen werden höchstens die ersten fünf Zeichen in Großschreibung verwendet. So entstehen beispielsweise `CONFI`, `FRAME`, `BASIS`, `PKA` und `AUTON`. Zwei Projekte eines Repositorys dürfen dabei nicht denselben Projektcode ergeben.

Als aktueller fachlicher Referenzstand gelten folgende Projekte:

Repository	Mandantenkürzel	Projekte	CodePipeline-Subsystem
mtext-fi	FI	Configuration, Fonts, LOMS_Framework, LOMS_Basis, LOMS_PKA	LOMS
mtext-autonom	IT	LOMS_Autonom	ITMT
mtext-by	BY	LOMS_Basis[BY], LOMS_Autonom[BY]	BYMT
mtext-lh	LH	LOMS_Basis[LH], LOMS_Autonom[LH]	LHMT
mtext-nw	NW	LOMS_Basis[NW], LOMS_Autonom[NW]	NWMT
mtext-os	OS	LOMS_Basis[OS], LOMS_Autonom[OS]	OSMT
mtext-sa	SA	LOMS_Basis[SA], LOMS_Autonom[SA]	SAMT

Diese Matrix schreibt den Lieferumfang technisch nicht fest. Eine fachlich abgestimmte Änderung der Projektstruktur bleibt möglich. Der Config-Check gibt gegenüber diesem Referenzstand fehlende oder zusätzliche Projekte im Workflow-Protokoll mit dem Präfix **WARNUNG:** aus. Einen ansonsten gültigen Stand verarbeitet er weiter.

4.1.5.2 Mandantenspezifische Werte für die technische Übergabe

Die technische Übergabe an die IZE9 benötigt die ISPW/CodePipeline-Instanz (P(roduktion) oder T(est)), das Assignment und die CodePipeline-Stage. Diese Werte werden in `.github/config.json` versioniert und können aufgrund besonderer Erfordernisse angepasst werden. Da die Konfiguration Bestandteil eines Branches ist, ist es z.B. möglich unterschiedliche Assignments je Release zu nutzen.

Beispiel:

```
"ispw": "P",
"hostprofile": {
  "FKT": {
    "assignment": "LOMS0000066",
    "stage": "FKTE"
  },
  "JUR": {
```

```

    "assignment": "LOMS0000067",
    "stage": "JURP"
  }
}

```

CodePipeline Elemente werden gemäß Mandantenkürzel, maximal 5-stelliger Projektcode (aus Projektverzeichnis-Name) und einem Suffix **F** oder **D** (FULL oder DELTA) benannt. So wird beispielsweise aus dem Tonic-Fragmentprojekt `LOMS_Autonom[BY]` das Element `BYAUTONF` bzw. `BYAUTOND`. Die zugehörige Archivdatei erhält zusätzlich die Endung `.tgz`. Ein F-Element enthält den vollständigen Projektbaum. Ein reguläres D-Element enthält die kumulativ seit dem `.100`-Release neuen und geänderten Dateien sowie die Löschliste. Bei einem `.100`-Release entsteht zusätzlich zum F-Element ein leeres D-Element mit leerem Projektverzeichnis und leerer Löschliste. Die Informationsdatei `_INFO_...txt` gehört zum Releasebeleg und wird nicht als CodePipeline-Element registriert.

Ein Push mit einer Änderung an `.github/config.json` startet automatisch **Validate mandant configuration**. Dieser Check prüft die Datei technisch und liefert frühzeitig eine verständliche Fehlermeldung. Er erkennt ein unbekanntes Mandantenkürzel, ein unpassendes Repository, fehlende oder ungültige Hostprofile und mehrdeutige abgeleitete Projektcodes.

4.1.6 6. Neue Releaselinie einrichten

Für jedes neue Release `Rnnn` werden die drei Branches `Rnnn/Entwicklung`, `Rnnn/Abnahme` und `Rnnn/Bereitstellung` von den Repository-Verantwortlichen angelegt. Ausgangspunkt ist normaler Weise der zuletzt gelieferte Release-Tag der bisherigen Linie.

Für das neue Release wird der älteste Eintrag in `config/releaselinien.json` aktualisiert mit der neuen Zuordnung aus Releaselinie, ETAPS-Linie und Hostprofil.
Beispiel:

```

{
  "R260": {"etaps_linie": "en03", "hostprofil": "JUR"},
  "R261": {"etaps_linie": "en01", "hostprofil": "FKT"},
  "R270": {"etaps_linie": "en02", "hostprofil": "JUR"}
}

```

Bei Änderungen an den Releaselinien, welche zentral durch die FI durchgeführt werden, müssen die Mandanten-Workflows anschließend einmal upgedated werden, damit sie auf eine Version der zentralen Workflows zeigen, die die aktualisierten Releaselinien enthält. Ein Vorteil dieser Lösung ist, dass Bereitstellungen immer absolut nachvollziehbar bleiben. Sowohl die verwendete Mandantenkonfiguration und die gesamte Tonic-Projektstruktur als auch die zum Releasezeitpunkt

gültigen zentralen Elemente wie Releaselinien-Konfiguration und verwendete Workflows bilden eine zusammengehörige und unveränderliche Version.

4.1.7 7. Änderung nach Entwicklung bringen

1. In der M/Text Workbench prüfen, dass der aktuelle `<Releaselinie>/Entwicklung` geöffnet ist. Für länger laufende oder gemeinsam bearbeitete Änderungen kann – nach Freigabe dieses Bedienwegs – ein Feature-Branch verwendet werden.
2. Die fachlich vorgesehenen Änderungen an den Briefressourcen durchführen.
3. Im integrierten Git-Client nur die vorgesehenen Änderungen für den Commit auswählen und eine aussagekräftige Commit-Nachricht eintragen.
4. Commit und Push auf den vorgesehenen Entwicklungsbranch ausführen.
5. In GitHub unter **Actions** → **Sync M/Text resources** den durch den Push ausgelösten Lauf kontrollieren.

Der Push startet die Synchronisation des exakten Commits mit dem aus Releaselinie und Stage zentral ermittelten M/Text-Ziel. Für R261 sind dies beispielsweise `en01e` in Entwicklung und `en01a` in Abnahme. Ein erfolgreicher Lauf endet mit `ADAPTER_ACCEPTED`.

Nach mehreren kurz aufeinanderfolgenden Pushes ist in GitHub zu prüfen, dass der letzte erfolgreiche Sync-Lauf den aktuellen Commit des Stage-Branches verarbeitet hat. Maßgeblich ist der gewünschte vollständige Zielstand, nicht die erfolgreiche Verarbeitung jedes zwischenzeitlichen Commits.

4.1.8 8. Stand zur Abnahme weitergeben

1. Die in Entwicklung erfolgreich geprüften Commits ermitteln.
2. Den aktuellen Zielbranch `<Releaselinie>/Abnahme` auschecken und aktualisieren.
3. Die freigegebenen Commits per Cherry-Pick aus `<Releaselinie>/Entwicklung` übernehmen.
4. Die neu entstandenen Commits und ihre Änderungen kontrollieren und den Abnahmebranch einmal pushen.
5. Unter **Actions** → **Sync M/Text resources** den Abnahmelaufl kontrollieren.
6. Die fachliche Abnahme außerhalb des Workflows nach dem vereinbarten Verfahren dokumentieren.

Der Push nach Abnahme verteilt den vollständigen Projektstand des erreichten Ziel-Commits. Er baut noch kein Mainframe-Paket.

4.1.9 9. Ausgewählte Änderungen bereitstellen

1. In GitHub die fachlich freigegebenen Commits aus Abnahme bestimmen und, falls Abhängigkeiten bestehen, ihre Reihenfolge festhalten.

2. Den aktuellen Zielbranch `<Releaselinie>/Bereitstellung` auschecken und aktualisieren.
3. Das Mandanten-Release-Team übernimmt diese Commits per Cherry-Pick aus `<Releaselinie>/Abnahme`.
4. Den resultierenden Stand und die neu entstandenen Commits kontrollieren und den Bereitstellungsbranch ohne Force-Push pushen.
5. Den exakten Ziel-Commit in GitHub prüfen und notieren. Der Push startet weder Paketbau noch Mainframe-Übergabe.

Nur das für den jeweiligen Mandanten benannte Mandanten-Release-Team darf hier pushen.

4.1.10 10. FULL- oder DELTA-Lieferung auslösen

Vor dem Taggen müssen Releaselinie, Mandant, gewünschter Lieferungstyp und der exakte Commit auf `<Releaselinie>/Bereitstellung` fachlich bestätigt sein.

- `Rnnn.100` erzeugt je nicht ausgeschlossenen Projekt ein F-Paket mit dem vollständigen Stand und zusätzlich ein leeres D-Paket mit leerem Projektverzeichnis und leerer Löschliste.
- Jede andere gültige dreistellige Endung, zum Beispiel `R261.108`, erzeugt ein kumulatives DELTA von `R261.100` bis zum Zieltag.
- Ein DELTA setzt voraus, dass der `.100`-Tag derselben Linie vorhanden und in der Git-Historie ein Vorgänger des Ziel-Tags ist.
- Nur das benannte Mandanten-Release-Team darf einen Release-Tag setzen oder löschen.

Der Release-Tag wird vom Mandanten-Release-Team als reiner Git-Tag angelegt. GitHub Releases mit Titel, Release Notes oder zusätzlichen Dateien werden nicht verwendet:

1. Den aktuellen Branch `<Releaselinie>/Bereitstellung` im zusätzlichen Git-Client auschecken und den neuesten GitHub-Stand abrufen.
2. Die vollständige SHA des bestätigten Ziel-Commits erneut vergleichen.
3. Den neuen Tag, beispielsweise `R261.108`, genau auf diesem Commit anlegen.
4. Ausschließlich diesen Tag nach GitHub pushen.
5. In GitHub prüfen, dass der Tag auf die bestätigte Commit-SHA zeigt und einen Lauf von **Build and publish release** gestartet hat.

Das Pushen des Git-Tags ist die fachliche Freigabe. Es startet den Paketbau und nach erfolgreicher Prüfung automatisch die Übergabe an die IZE9.

Danach unter **Actions** → **Build and publish release** prüfen:

1. `Build FULL or DELTA artifact` validiert Konfiguration, Tag und Branchzuordnung, baut die Pakete und erzeugt `manifest.json` mit SHA-256-Prüfsummen. Bei `.100` müssen je Projekt sowohl das F-Paket als auch das zusätzliche leere D-Paket enthalten sein.

2. Das Artefakt heißt `<Repository>-<Release-Tag>` und wird standardmäßig 30 Tage aufbewahrt.
3. `Verify and hand over artifact to Mainframe` startet nach erfolgreichem Build automatisch und bindet das Environment `Bereitstellung`.
4. Vor der Übergabe werden Pfad, Größe und SHA-256 jeder manifestierten Datei geprüft. Danach werden die JCL-Werte validiert und ins Template gerendert, und dann das Paket plus JCL per FTP/JES an die IZE9 übergeben.

4.1.10.1 Irrtümlichen Tag zurücknehmen

Wurde der Tag auf dem falschen Commit oder mit dem falschen Namen angelegt, bricht das Mandanten-Release-Team einen noch laufenden Ablauf ab. Anschließend löscht es den irrtümlichen Git-Tag lokal und auf GitHub und legt bei Bedarf den richtigen Tag an. Der neu angelegte Tag startet einen neuen Lauf.

Eine bereits erfolgte Mainframe-Übergabe wird durch das Löschen eines Tags nicht rückgängig gemacht. Der korrigierte Lauf überschreibt die betreffenden Member und CodePipeline versioniert den neuen Stand.

`MAINFRAME_SUBMITTED` bedeutet ausschließlich, dass die unmittelbare FTP-/JES-Übergabe akzeptiert wurde. Der fachliche Abschluss des Mainframe-Jobs wird in Ausbaustufe 1 nicht abgefragt. Das Mandanten-Release-Team prüft den weiteren Status selbst auf dem Host nach dem dafür festgelegten Betriebsverfahren.

4.1.11 11. Einen Lauf kontrolliert wiederholen

Ein noch verfügbarer Workflow-Lauf kann in GitHub über **Actions**, den betroffenen Lauf und **Re-run jobs** mit demselben Commit und derselben Git-Referenz wiederholt werden. Die folgenden manuellen Durchführungen der Workflows sind für Fälle gedacht, in denen ein neuer kontrollierter Lauf mit ausdrücklich angegebenem Commit oder Tag erforderlich ist.

4.1.11.1 M/Text-Vollsynchronisation initial starten oder wiederholen

Unter **Actions** → **Sync M/Text resources** → **Run workflow** angeben:

- `commit_sha` : vollständiger 40-stelliger SHA des bereits geprüften Commits,
- `source_branch` : der passende Branch `Rnnn/Entwicklung` oder `Rnnn/Abnahme`.

Die Automation weist den Lauf zurück, wenn der Commit nicht aus dem gewählten Branch erreichbar ist. Das Zielsystem wird aus dem Branch abgeleitet.

Ein Wiederanlauf eines älteren Sync-Laufs kann einen inzwischen neueren M/Text-Zielstand wieder durch den vollständigen Stand des älteren Commits ersetzen. Vor **Re-run jobs** oder **Run workflow** müssen deshalb der angegebene Commit, der aktuelle Branch-Commit und der gewünschte Zielstand verglichen werden. Ist der Branch inzwischen weitergelaufen, wird nicht der alte Lauf wiederholt, sondern ausschließlich der aktuell gewünschte Commit kontrolliert synchronisiert.

4.1.11.2 Release-Lauf wiederholen

Unter **Actions** → **Build and publish release** → **Run workflow** den bereits existierenden `release_tag` angeben. Keinen neuen Tag nur für einen technischen Wiederanlauf erzeugen. Die Automation löst den Tag erneut auf und prüft seine Erreichbarkeit aus `<Releaselinie>/Bereitstellung`.

Vor jedem Wiederanlauf klären, ob das Zielsystem die vorherige Übergabe bereits angenommen hat. GitHub Actions kennt ohne Status-Polling keinen nachgelagerten fachlichen Endstatus.

4.1.12 12. Status und Fehlerbilder verstehen

Status oder sichtbares Symptom	Bedeutung	Nächste Prüfung
Workflow kann die <code>mtext-actions</code> -Version nicht laden	Technische Einrichtung ist unvollständig oder der Zugriff auf <code>mtext-actions</code> fehlt	Zentrale Automatisierungsverantwortliche informieren. Anwender ändern die Workflowdateien nicht selbst
<code>CONFIG_VALIDATED</code>	Mandantenkonfiguration und Releaselinienzuordnung sind technisch konsistent	Fachliche Abstimmung fortsetzen. Der Status ist keine automatische Freigabe für die nächste Stage
<code>VALIDATION_FAILED</code>	Eingabe, Konfiguration, Branchrichtung oder JCL ungültig	Erste Fehlermeldung lesen. Branch- oder Tagformat und Konfiguration prüfen
<code>SOURCE_FAILED</code>	Commit oder Tag/Basisreferenz nicht auflösbar	SHA, Tag, <code>.100</code> -Basis und Branch-Erreichbarkeit prüfen
<code>PACKAGE_FAILED</code>	Paket, Informationsdatei oder manifestiertes Artefakt konnte nicht erstellt beziehungsweise geprüft werden	Fehlende Projekte, Symlinks, vorhandenes Ausgabeverzeichnis sowie Pfad, Größe und SHA-256 der Artefakte prüfen
<code>RESOURCE_TRANSFER_FAILED</code>	Staging oder Veröffentlichung nach <code>serverSync</code> fehlgeschlagen	Runner-Dateisystem und Share-/NFS-Ziel einschließlich vorhandener temporärer Verzeichnisse prüfen
<code>ADAPTER_FAILED</code>	Transportfehler oder Nicht-2xx vom Adapter	HTTP-Status und gekürzten Antworttext im Protokoll prüfen

Status oder sichtbares Symptom	Bedeutung	Nächste Prüfung
ADAPTER_ACCEPTED	Unmittelbare Adapterannahme erfolgreich	Kein Beleg für einen nachgelagerten M/Text-Endstatus. Ist die fachliche Wirkung unklar, wird die Anwendungsbetreuung eingeschaltet. Anwender benötigen keinen direkten Zugriff auf technische Anwendungsprotokolle
ARTIFACT_READY	Releaseartefakt lokal gebaut und geprüft	Automatisch gestarteten Publish-Job prüfen
MAINFRAME_TRANSFER_FAILED	FTP-Upload oder JES-Übergabe unmittelbar fehlgeschlagen	Zugangsdaten, Dataset, JES-Ziel, Netzwerk und FTP-Antwort prüfen
MAINFRAME_SUBMITTED	Unmittelbare FTP-/JES-Übergabe akzeptiert	Das Mandanten-Release-Team prüft den weiteren Status selbst auf dem Host nach dem festgelegten Betriebsverfahren

Protokolle dürfen keine Zugangsdaten enthalten. Zugangsdaten niemals in Konfigurationsdateien, Workflow-Eingaben, GitHub-Kommentare oder Support-Tickets kopieren.

5 GitHub Repositories, Teams und IDNeo Fachrollen

5.1 Allgemein

Für den Zugriff auf die zukünftigen GitHub-Repositories in der FI sollten in der jeweiligen OE Fachrollen definiert und diese den entsprechenden Komponentenrollen zugeordnet werden.

Wie die Fachrollen strukturiert werden, bleibt uns überlassen (Komponente: GitHub Enterprise).

Es gibt pro Repository grundsätzlich vier Berechtigungsstufen:

_maintain
_triage
_write
_read

Die jeweilige Bedeutung der Rechte ist hier ausführlich beschrieben: [Berechtigungsstufen für Repositories "in a Nutshell"](#)⁶



Die Berechtigungsstufen sind kumulativ. So beinhaltet z.B. **maintain** auch die Rechte von **write**, **triage** und **read**. Ein Nutzer benötigt also lediglich *eine* Komponentenrolle, nämlich die *seiner* höchsten Berechtigungsstufe.

Eine Komponentenrolle in IDNeo würde dann für den Maintainer

Repo `fi_lbs_entw_oms_mtext_actions` bspw. so aussehen:

`Github-Produktion_fi_lbs_entw_oms_mtext_actions_maintain`

5.2 Übersicht

Die folgenden Repositories und Komponentenrollen wurden angelegt:

Komponentenrolle	Repository	Zweck	Bereich
Github-Produktion_FinanzInformatik_fi_lbs_entw_oms_fi	fi_lbs_entw_oms_fi	Master für integrierte Briefe, M/OMS + M/Text Konfiguration, Framework Projekt	LBS OMS

6. <https://wiki.intern/spaces/admingit/pages/444323573/Berechtigungsstufen+f%C3%BCr+Repositories+in+a+Nutshell>

Komponentenrolle	Repository	Zweck	Bereich
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_it	fi_lbs_entw_oms_it	Master autonome Briefe	LBS OMS
Github-Produktion_fi_lbs_entw_oms_bw	fi_lbs_entw_oms_bw	Individuelle Brief-Ressourcen LBS SüdWest	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_by	fi_lbs_entw_oms_by	Individuelle Brief-Ressourcen LBS Süd	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_nw	fi_lbs_entw_oms_nw	Individuelle Brief-Ressourcen LBS NordWest	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_lh	fi_lbs_entw_oms_lh	Individuelle Brief-Ressourcen LBS Hessen-Thüringen	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_os	fi_lbs_entw_oms_os	Individuelle Brief-Ressourcen LBS NordOst	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_sa	fi_lbs_entw_oms_sa	Individuelle Brief-Ressourcen LBS Saar	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_mtext_adapter	fi_lbs_entw_oms_mtext_adapter	Sourcen von LTOMA	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_mtext_actions	fi_lbs_entw_oms_mtext_actions	Zentrale GitHub Actions Workflows	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_mserie	fi_lbs_entw_oms_mserie	?	LBS OMS
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_mserie_support	fi_lbs_entw_oms_mserie_support	Issue-Tracker	LBS OMS

Komponentenrolle	Repository	Zweck	Bereich
GitHub-Produktion_FinanzInformatik_fi_lbs_entw_oms_testauto	fi_lbs_entw_oms_testauto	Testautomatisierung	LBS OMS

5.3 GitHub Einrichtung

[Git Client konfigurieren - Authentifizieren und Signieren - Administration GitHub - Wiki-System](#)⁷

7. <https://wiki.intern/spaces/admingit/pages/456079199/Git+Client+konfigurieren+-+Authentifizieren+und+Signieren>

6 Nächste Schritte (OPL)

6.1 1. Technische Grundlagen festlegen

N r.	Status	Tätigkeit	Ergebnis
1.1	BESTÄTIGT	GitHub-Plattform festhalten	GitHub Enterprise Server 3.20.4 ist die voraussichtliche Zielplattform
1.2	OFFEN	Repositories anlegen	<team>/mtext-actions und <team> /mtext-fi sind angelegt (Teams wurden definiert, Berechtigungen via idneo vergeben) - alle weitere Mandanten-Repositories und das Issue-Tracker Repository mtext-support wurden angelegt
1.3	BESTÄTIGT	mtext-actions gegen den fachlichen Vertrag prüfen	- Die Akzeptanztests für workflow_configuration und die vier fachlichen CLI-Kommandos sind erfolgreich (vorab getestet) - FULL und DELTA, Artefaktprüfung, JCL, FTP-/JES-Übergabe, Ressourcensynchronisation und Workflowgrenzen sind abgedeckt (vorab getestet)
1.4	OFFEN	Mandantenspezifische Verantwortliche benennen	Für jeden Mandanten sind die relevanten Personen namentlich zugeordnet, User eingerichtet und zugewiesen

6.2 2. Runnerangebot der FI bereitstellen und prüfen

N r.	Status	Tätigkeit	Ergebnis
2.1	OFFEN	Runner der FI bereitstellen	- Der Runner kann genutzt werden - Sein bestätigtes runs-on-Kennzeichen ist in mtext-actions als Repositoryvariable FI_RUNNER_LABEL hinterlegt.
2.2	OFFEN	Einheitliche Laufzeitvorbereitung bestätigen	Python ist in der geforderten Version verfügbar (mind. 3.11)

N r.	Status	Tätigkeit	Ergebnis
2.3	OFFEN	GHES-Artefakt-Actions prüfen	• <code>upload-artifact</code> v3.2.2 läuft auf GHES 3.20.4 erfolgreich • <code>download-artifact</code> v3.1.0 läuft auf GHES 3.20.4 erfolgreich

6.3 3. GitHub-Einstellungen und Berechtigungen einrichten

N r.	Status	Tätigkeit	Ergebnis
3.1	OFFEN	Technische Einrichtungsberechtigung hinterlegen	• Das Environment <code>Einrichtung</code> enthält <code>WORKFLOW_CONFIGURATION_TOKEN</code> • Der Zugriff auf Mandanten-Repositories ist auf die freigegebenen Branches begrenzt
3.2	OFFEN	Stage-Branches und Default Branch einrichten	• Für <code>R260</code>, <code>R261</code> und <code>R270</code> bestehen jeweils <code>Entwicklung</code>, <code>Abnahme</code> und <code>Bereitstellung</code>. • <code>R261/Entwicklung</code> ist als erster Default Branch eingestellt.
3.3	OFFEN	Repositoryübergreifenden Zugriff auf <code>mtext-actions</code> einrichten	• Die vorgesehenen Mandanten-Repositories können die wiederverwendbaren Workflows aus <code>mtext-actions</code> aufrufen. • Der technische Checkout der Python-Implementierung aus <code>mtext-actions</code> ist ausschließlich lesend. • Workflowaufruf und Checkout wurden praktisch geprüft.
3.4	OFFEN	Branchschutz für die drei Stages einrichten	• Berechtigte Text-Entwickler können nach <code>Entwicklung</code> und <code>Abnahme</code> pushen • Nur das Mandanten-Release-Team kann nach <code>Bereitstellung</code> pushen • Nicht berechtigte Pushes werden abgewiesen • Löschen und Force-Pushes werden auf allen Stage-Branches abgewiesen

N r.	Status	Tätigkeit	Ergebnis
3.6	OFFEN	Workflowdateien und Mandantenkonfiguration schützen	• Normale Ressourcen-Pushes mit Änderungen an <code>.github/workflows/**/*</code> werden abgewiesen • Änderungen an <code>.github/config.json</code> sind auf den technischen Konfigurationskreis begrenzt
3.7	OFFEN	GitHub Environments konfigurieren	• Die Environments <code>Einrichtung</code> und <code>Bereitstellung</code> besitzen die vorgesehenen Branch-/Tagregeln • Der Publish-Job kann <code>Bereitstellung</code> verwenden
3.8	OFFEN	Schutz der Release-Tags praktisch testen	• Ein Test hält fest, dass ein Tag wie <code>R261.108</code> in GitHub technisch so geschützt werden kann, dass nur das Mandanten-Release-Team sie erstellen, verändern oder löschen darf
3.9	OFFEN	Rücknahme irrtümlicher Release-Tags abnehmen	• Das Mandanten-Release-Team kann einen irrtümlichen Tag löschen • Vor dem Löschen wird der dadurch gestartete Workflow-Lauf abgebrochen • Ein neu angelegter Tag startet einen neuen Release-Workflow

6.4 4. Workflowdateien einrichten und aktualisieren

Der Workflow **Configure workflow files** aus `configure-workflows.yml` bearbeitet pro Lauf einen Mandantenbranch. Das Python-Modul `workflow_configuration` bereitet die geprüften Commits vor. Der Workflow pusht sie erst nach erfolgreicher Abschlussprüfung.

N r.	Status	Tätigkeit	Ergebnis
4.1	OFFEN	Configure workflow files abnehmen	• Der Workflow aktualisiert <code>mtext-fi</code> wie gewünscht und die Workflows in <code>mtext-fi</code> nutzen dann <code>mtext-actions</code> korrekt
4.2	OFFEN	Restliche Mandanten-Workflows konfigurieren	• alle weiteren Mandanten-Repositories werden wie in 4.1 eingerichtet

6.5 5. M/Text-Transport entscheiden und abnehmen

N r.	Status	Tätigkeit	Ergebnis
5.1	OFFEN	Linien- und URL-Mapping einrichten	- R260 verwendet en03 , R261 verwendet en01 und R270 verwendet en02 - LTOMA und LTOMS sind jeweils unter https://en0[1,2,3][e,a].ltom[a,s].intern⁸ verfügbar
5.2	OFFEN	Transportweg nach serverSync festlegen	- Eine der im Zielbild beschriebenen Varianten ist ausgewählt und implementiert - Erfolgsprüfung, Fehlergrenzen und Wiederanlauf sind festgelegt
5.3	OFFEN	Zertifikate und M/Text-Netzwerkpfade prüfen	- Die interne CA ist im Truststore des Runners vorhanden - Der Runner erreicht die ausgewählten M/Text-Ziele
5.4	OFFEN	Kompatibilität des gewählten Transports nachweisen	Dateien und Verzeichnisse unter serverSync entsprechen dem bisherigen Verfahren
5.5	OFFEN	Fehler- und Wiederanlaufverhalten abnehmen	- Ein Abbruch während Übertragung oder Veröffentlichung startet keine interne Synchronisation - Ein Wiederanlauf stellt den vollständigen Stand jedes verarbeiteten Projekts her - Für vollständig entfernte oder neu ausgeschlossene zusätzliche Projekte ist eine sichere Bereinigungsregel festgelegt, die keine Verzeichnisse anderer Mandanten-Repositories berührt
5.6	OFFEN	Config-Check und Synchronisation prüfen	- Der Config-Check endet mit CONFIG_VALIDATED - Der vollständige Projektstand eines festgelegten Commits wird über den gewählten Transport bereitgestellt - Der Wiederanlauf desselben Commits ist erfolgreich

8. [https://enXXe.ltom\[a,s\].intern](https://enXXe.ltom[a,s].intern)

6.6 6. Mainframe-Übergabe einrichten und abnehmen

N r.	Status	Tätigkeit	Ergebnis
6.1	OFFEN	FTP-Secrets hinterlegen	- Bereitstellung enthält MAINFRAME_FTP_HOST, MAINFRAME_FTP_USER und MAINFRAME_FTP_PASSWORD - Die Einrichtungsprüfung bestätigt nur das Vorhandensein der Secret-Namen
6.2	OFFEN	Mainframe-Netzwerkpfade prüfen	Der Runner erreicht Mainframe-FTP und JES
6.4	OFFEN	Historischen trans - Vertrag abnehmen	- Paketnamen und Inhalte, FULL- und DELTA-Pfade entsprechen den Referenzlieferungen - Ein FULL erzeugt je Projekt das vollständige F-Paket und das zusätzliche leere D-Paket
6.5	OFFEN	JCL und unmittelbare Übergabe prüfen	- Die konfigurierte ISPW-Instanz erscheint korrekt in Dataset und Programmaufruf - FTP-Anmeldung, Übertragung und SITE FILETYPE=JES sind erfolgreich - Das gerenderte JCL wird erfolgreich übergeben

6.7 7. Git-Client und Bedienabläufe abnehmen

N r.	Status	Tätigkeit	Ergebnis
7.1	OFFEN	Funktionalität des Git-Clients in der Workbench abgenommen	Die Fähigkeiten des integrierten Clients sind geprüft und dokumentiert (Bedienungsanleitung)
7.2	OFFEN	Ressourcenarbeit, Stage-Weitergabe und Rücknahme abnehmen	- Status, Änderungen, Commit und Push sind in den vorgesehenen Clients praktisch geprüft - Ein Cherry-Pick kann bei einem Konflikt fortgesetzt oder vollständig abgebrochen werden - Die Abläufe entsprechen der Benutzeranleitung

6.8 8. Test-Parallelbetrieb vorbereiten

N r.	Status	Tätigkeit	Ergebnis
8.1	OFFEN	Importumfang und Release-Basen freigeben	- Aktive Releaselinien, Projekte, Dateinamen und Ausschlüsse sind je Mandant inventarisiert - Je aktiver Releaselinie sind mindestens der .100 -Stand und alle späteren als Releases benötigten Tags zur Übernahme benannt
8.2	OFFEN	Ersten SVN-Abzug erstellen	- Der Abzug folgt dem Migrations- und Cutover-Runbook (see page 47) - Alle Mandanten-Repositories besitzen einen dokumentierten SVN-Ausgangsstand für die Tests
8.3	OFFEN	Nichtproduktiven End-to-End-Pilot abnehmen	- Git-Clients und Schutzregeln sind praktisch geprüft - M/Text-Synchronisation und Wiederanlauf sind erfolgreich - FULL- und DELTA-Release einschließlich Mainframe-Übergabe sind erfolgreich
8.4	OFFEN	Mandanten konnten eigene Abläufe erfolgreich testen	Vorbereitende Tätigkeiten der Mandanten konnten erfolgreich durchgeführt und abgenommen werden